PERBANDINGKAN KINERJA ALGORITMA GMM DAN TDNN DALAM PENGENALAN WICARA BAHASA JAWA

PROPOSAL SKRIPSI

Diajukan untuk Memenuhi Salah Satu Syarat Memperoleh Gelar Sarjana

Komputer Program Studi Informatika

Diajukan Oleh:

JOSEPH SERI APRILLIO DE ORNAY

225314118

Progam Studi Informatika

Fakultas Sains dan Teknologi

Universitas Sanata Dharma

Yogyakarta

2025

HALAMAN PERSETUAN PEMBIMBING

PROPOSAL SKRIPSI

PERBANDINGKAN KINERJA ALGORITMA GMM DAN TDNN DALAM PENGENALAN SUARA BAHASA JAWA

Proposal Skripsi yang diajukan oleh:

JOSEPH SERI APRILLIO DE ORNAY

NIM : 225314118

Telah disetujui oleh:

Dosen Pembimbing :

Tanggal……..

Drs. Hari Suparwito, S.J., M.App.IT

DAFTAR ISI

DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1. Latar Belakang. 1
1.2. Rumusan Masalah. 3
1.3. Tujuan Penelitian. 3
1.4. Manfaat Penelitian. 3
1.5. Batasan Masalah. 4
BAB II TINJAUAN PUSTAKA 5
2.1 Tinjauan Pustaka. 5
2.2 Landasan Teori. 10
2.2.1. Data Suara. 10
2.2.2. Kompresi Berkas Suara. 11
2.2.3. Format Berkas Data Suara. 11
2.2.4. Automatic Speech Recognition. 12
2.2.5. Stratified Sampling. 13
2.2.6. Preprocessing. 13
2.2.7. Data Splitting. 14
2.2.8. Feature Extraction. 14
2.2.9. Acoustic Model. 19
2.2.10. Gaussian Mixture Model. 21
2.2.11. Lexicon. 21
2.2.12. Hidden Markov model (HMM) 26
2.2.13. Deep Neural Network. 21
2.2.14. Time Delay Neural Network. 23
2.2.15. ReLU. 25
2.2.16. Softmax. 25
2.2.17. Language Model 27
2.2.18. N-Gram. 28
2.2.19. Viterbi. 32
2.2.20. *Hyperparameter Tunning*. 33
2.2.21. Word Error Rate (WER). 34
BAB III METODE PENELITIAN 35
3.1. Alur Proses Penelitian. 35
3.2. Data. 36

3.3. Metode Penelitian..... 37
3.4. Desain Penelitian..... 80
Daftar Pustaka..... 82

BAB I PENDAHULUAN

1.1.Latar Belakang.

Perkembangan teknologi dari waktu ke waktu telah mengubah cara manusia menangani persoalan sehari-hari. *Automatic Speech Recognition* (ASR) atau pengenalan ucapan otomatis merupakan sistem yang dirancang untuk mengidentifikasi dan menerjemahan ucapan manusia menjadi teks. Tanpa kita sadari, ASR telah mengambil peran dalam kehidupan kita sehari-hari. ASR menjadi terobosan yang memungkinkan adanya alat kontrol suara, hingga transkripsi audio menjadi teks secara otomatis. Teknologi ASR menjadi basis dari teknologi asisten virtual yang sering kita jumpai seperti Siri, Alexa, Google Assistant, dan fitur voice typing di berbagai aplikasi. Tak hanya itu, Layanan pelanggan otomatis menggunakan asisten cerdas serta sistem navigasi pada kendaraan berbasis perintah suara juga dibangun berbasis teknologi ASR. (Mohammad, 2020; Bhatt et al, 2020).

Automatic Speech Recognition (ASR) merupakan teknologi yang diterapkan luas pada setiap segmen kehidupan (Iosifova et al, 2021). Namun, penerapan Automatic Speech Recognition (ASR) pada umumnya masih berfokus pada bahasa utama seperti, bahasa Inggris dan bahasa Indonesia. Kondisi ini disebabkan oleh ketersediaan sumber daya untuk bahasa utama lebih besar dibandingkan bahasa lokal seperti bahasa Jawa. Bahasa Jawa merupakan bahasa daerah dengan penutur terbanyak di Indonesia (Arisaputra et al, 2024). Penggunaan bahasa Jawa dapat dijumpai dari komunikasi dalam kehidupan sehari-hari, siaran media lokal, konten digital, hingga kegiatan budaya seperti upacara adat hingga seni pertunjukan. (Setiawan, 2025; Mardikantoro, 2016; Udasmoro et al, 2023) Selain kekurangan ketersediaan sumber daya dibandingkan bahasa utama, karakteristik dan kompleksitas dari bahasa daerah juga menjadi tantangan tersendiri dalam penerapan ASR. Karakteristik bahasa daerah lebih

kompleks karena memiliki struktur bahasa yang berebeda, serta kosakata yang kaya (Arisaputra et al, 2024).

Sebuah sistem Automatic Speech Recognition (ASR) atau pengenalan ucapan otomatis terdiri atas 2 model, yakni model akustik (*Acoustic Model*) dan model bahasa (*Language Model*). Model akustik berfungsi untuk mengonversi suara menjadi representasi fonem, sedangkan model bahasa digunakan untuk melakukan kalkulasi probabilitas dari suatu kata dalam kalimat agar sesuai dengan konteks. Sistem ASR membutuhkan kedua model, terutama model akustik ketika berhubungan dengan konteks dari fitur ucapan. Model akustik seperti kombinasi *Gaussian Mixture Model* (GMM) dan *Hidden Markov Model* (HMM) sudah menjadi standar model akustik sebelum masuknya pembelajaran mendalam (Deshmukh, 2020; Atmaja et al, 2018). Pembelajaran mendalam memberikan salah satu pendekatan yang popular yakni *Time Delay Neural Network* (TDNN) (Gunarso et al,2024). Mengingat tantangan pada bahasa daerah seperti bahasa Jawa dengan perbedaan pola data. Penelitian ini berfokus membandingkan model akustik *Gaussian Mixture Model* (GMM) dengan model akustik pembelajaran mendalam seperti *Time Delay Neural Network* (TDNN).

Penelitian Gunarso et al. (2024) menegaskan bahwa pendekatan *deep learning* lebih unggul dari GMM-HMM, dengan model TDNN-nya berhasil mencapai *Word Error Rate* (WER) sebesar 12,16% untuk pengenalan wicara bahasa Indonesia. Penelitian Aung et al. (2020) menunjukkan bahwa algoritma TDNN secara signifikan mengungguli GMM-HMM (26,11% WER), dan mampu mengenali ucapan dalam bahasa Myanmar dengan kinerja terbaik, mencapai tingkat *Word Error Rate* (WER) serendah 15,03% . Penelitian Farooq et al. (2019) menunjukkan bahwa model TDNN secara substansial mengungguli GMM-HMM (32,24% WER), dan berhasil mengenali ucapan kosakata besar dalam bahasa Urdu dengan tingkat Word Error Rate (WER) 21,33%. Penelitian yang akan dilakukan bertujuan untuk memberikan analisis terkait perbedaan kinerja model *baseline* GMM-HMM dan TDNN terhadap dataset bahasa Jawa.

1.2.Rumusan Masalah.

Bahasa Jawa merupakan bahasa dengan penutur terbanyak di Indonesia namun termasuk dalam bahasa *low-resource*, karena keterbatasan korpus yang membuat minimnya penelitian mengenai pengenalan wicara otomatis pada bahasa Jawa. Sehingga, mengakibatkan pengkajian model akustik untuk dataset bahasa Jawa masih terbatas. Maka dari itu, diperlukan pengujian model algoritma GMM dan TDNN terhadap dataset bahasa Jawa. Bagaimana ketepatan pengenalan wicara pada dataset bahasa Jawa menggunakan algoritma GMM dan TDNN ?

1.3.Tujuan Penelitian.

Tujuan penelitian yang ingin dicapai yakni :

1. Menerapkan algoritma GMM dan TDNN untuk melakukan pengenalan bahasa Jawa.
2. Menerapkan eksperimen desain dengan menggunakan metode *Hyperparameter Tunning.*
3. Mengukur kinerja GMM dan TDNN dalam melakukan pengenalan bahasa Jawa.
4. Menganalisis kelayakaan model baseline berdasarkan pengukuran kinerja pengenalan bahasa Jawa.

1.4. Manfaat Penelitian.

Penelitian mengenai perbedaan kinerja model akustik dapat menerapkan algoritma GMM dan TDNN untuk mengenali bahasa Jawa secara akurat.

1.5. Batasan Masalah.

Batasan masalah dalam penelitian ini sebagai berikut :

1. Dataset yang digunakan adalah rekaman suara bahasa Jawa yang telah dikumpulkan dalam kolaborasi Google dengan *Reykjavik University* dan Universitas Gadjah Mada. (https://www.openslr.org/35/)
2. Penelitian berfokus pada kinerja model akustik yakni algoritma GMM dan TDNN dalam pengenalan bahasa Jawa.

BAB II TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka.

Bahasa Jawa merupakan bahasa daerah dengan jumlah penutur yang terbanyak di Indonesia, namun hingga saat ini jarang ditemukan sistem pengenalan bahasa otomatis yang secara khusus difokuskan pada bahasa tersebut. Oleh karena itu, diperlukan kajian sebagai landasan dalam penerapan algoritma GMM dan TDNN untuk mengenali bahasa Jawa secara akurat. Penelitian Gunarso et al. (2024) menegaskan bahwa pendekatan *deep learning* lebih unggul dari model *baseline* GMM-HMM, dengan model TDNN-nya berhasil mencapai *Word Error Rate* (WER) sebesar 12,16% untuk pengenalan wicara bahasa Indonesia. Adapun penelitian yang dilakukan oleh Penelitian Aung et al. (2020) pada bahasa Myanmar dengan pendekatan *speaker-independent* menunjukkan bahwa model *baseline* GMM-HMM menghasilkan Word Error Rate (WER) sebesar 21,50% pada tahap *development*, 26,11% pada *TestSet1*, dan 28,78% pada *TestSet2*, sedangkan model Time Delay Neural Network (TDNN) memberikan hasil yang lebih baik dengan WER sebesar 11,25% pada tahap *development*, 15,03% pada *TestSet1*, dan 16,83% pada *TestSet2*. Penelitian Srivasta et al (2018) pada tiga bahasa yakni Tamil, Telugu, dan Gujarati. Pada bahasa Tamil model *baseline* GMM-HMM menghasilkan WER 33.55%, sedangkan algoritma TDNN menghasilkan WER yang lebih rendah yakni 19.45%. Pada bahasa Telugu model *baseline* GMM-HMM menghasilkan WER 40,12%, sedangkan algoritma TDNN menghasilkan WER yang lebih rendah yakni 22.61%. Pada bahasa Gujarati model *baseline* GMM-HMM menghasilkan WER 23.78%, sedangkan algoritma TDNN menghasilkan WER 19.76%. Penelitian lainnya dapat dilihat pada **Tabel 2.1** Hasil Perbandingan Penelitian.

Tabel 2.1 Hasil Perbandingan Penelitian.

Peneliti dan Tahun	Judul	Metode	Keterangan
Gunarso, Agus Buono, Mushthofa, M. Teduh Uliniansyah (Diterima: 14-06-2024; Disetujui: 14-08-2024; Volume 22 No. 1 Maret 2025)	Pengembangan model akustik dengan deep neural network untuk sistem pengenalan wicara bahasa Indonesia	TDNN	Time-Delay Neural Network (TDNN) (WER terbaik 12,16%)
Myat Aye Aye Aung, Win Pa Pa (2020)	Time Delay Neural Network for Myanmar Automatic Speech Recognition	GMM-HMM	Development (WER 21.50%), TestSet1(WER 26.11%), TestSet2 (WER 28.78%)
		TDNN	Development (WER 11.25%), TestSet1(WER 15.03%), TestSet2 (WER 16.83%)
Noor Fathima, Tanvina Patel, Mahima C and Anuroop Iyengar (2018)	TDNN-based Multilingual Speech Recognition System for Low Resource Indian Languages	GMM-HMM	Pendekatan campuran 3 bahasa (Gujarati, Tamil, Telugu) dengan %WER berturut turut 18,87 %,

			24,81%, dan 27,19%. Pendekatan campuran 2 bahasa (Tamil dan Telugu) dengan % WER beruturut turut (22.94 % dan 24.47 %) . Sedangkan hasil tanpa pendekatan campuran bahasa Gujarati memiliki % WER 17.28 %, Tamil 22.46 %, dan Telugu 24.21%.
		TDNN	Pendekatan campuran 3 bahasa (Gujarati, Tamil, Telugu) dengan %WER berturut turut 11,95 %, 17,32%, dan 18,84%. Pendekatan campuran 2 bahasa (Tamil dan Telugu)

			dengan % WER beruturut turut (16,46 % dan 17,69%) . Sedangkan hasil tanpa pendekatan campuran bahasa Gujarati memiliki % WER 11,58 %, Tamil 16,79%, dan Telugu 17,94%.
Brij Mohan Lal Srivastava, Sunayana Sitaram, Rupesh Kumar Mehta, Krishna Doss Mohan, Pallavi Matani, Sandeepkumar Satpal, Kalika Bali, Radhakrishnan Srikanth, Niranjan Nayak (2018)	Interspeech 2018 Low Resource Automatic Speech Recognition Challenge for Indian Languages	GMM-HMM	WER yang dihasilkan pada tiga bahasa yakni Tamil 33.55%, Telugu 40,12%, dan Gujarati 23.78%
		TDNN	WER yang dihasilkan pada tiga bahasa yakni Tamil 19.45%, Telugu 22.61%, dan Gujarati 19.76%
Fei Wu1, Leibny Paola Garcia, Daniel Povey,	Advances in Automatic Speech Recognition for	GMM	WER yang dihasilkan adalah 36.5 %

Sanjeev Khudanpur (2019)	Child Speech Using Factored Time Delay Neural Network	TDNN	WER yang dihasilkan adalah 15.8%
Boji Liu, Weibin Zhang, Xiangming Xu and Dongpeng Chen(2019)	Time Delay Recurrent Neural Network for Speech Recognition	TDNN	Terdapat tiga percobaan menggunakan TDNN sebagai pembanding utama. WER yang dihasilkan pada tahap development adalah 15.1%, WER yang dihasilkan pada tahap *Switchboard* adalah 12.1%, dan WER pada tahap hub5'00 adalah 17.9%.
Ganesh S Mirishkar, Aditya Yadavalli, Anil Kumar Vuppala (2021)	An Investigation of Hybrid architectures for Low Resource Multilingual Speech Recognition system in Indian context	GMM-HMM	Pada percobaan tipe mono menghasilkan rerata WER sebesar 46.88%
		TDNN	Pada percobaan tipe mono menghasilkan rerata WER sebesar 24.80%

Susmita Bhattacharjeea, Jagabandhu Mishrab, H.S. Shekhawata , S. R. Mahadeva Prasannac (2025)	Fairness of Automatic Speech Recognition in Cleft Lip and Palate Speech	GMM-HMM	WER yang dihasilkan sebesar 22.64%.
Hemant Kathania, Mittul Singh, Tamas Gr ´ osz, Mikko Kurimo (2020)	Data augmentation using prosody and false starts to recognize non-native children's speech	TDNN	WER yang dihasilkan sebesar 23.06 % untuk tahap *development* dan 20.26% untuk tahap *evaluate.*
Muhammad Umar Farooq, Farah Adeeba, Sahar Rauf, Sarmad Hussain	Improving Large Vocabulary Urdu Speech Recognition System using Deep Neural Networks	GMM	WER yang dihasilkan adalah 46.87%
		TDNN	WER yang dihasilkan adalah 21.33%

2.2 Landasan Teori.

2.2.1. Data Suara.

Suara merupakan fenomena fisik dari getaran suatu objek yang membentuk sinyal analog dengan amplitudo yang berfluktuasi secara kontinu terhadap waktu dan dinyatakan dalam satuan frekuensi. Suara direpresentasikan dalam bentuk gelombang dan disimpan oleh komputer menggunakan berbagai format berkas data suara. (Silitonga et al, 2019).

2.2.2. Kompresi Berkas Suara.

Kompresi atau pemadatan data merupakan suatu metode yang bertujuan untuk mengurangi ukuran data sehingga efisiensi penyimpanan dapat meningkat. Proses kompresi menghasilkan jumlah bit sekitar 26% hingga 63% lebih kecil dibandingkan format WAV 16-bit. Teknik kompresi dibedakan menjadi dua jenis yakni, kompresi tanpa kehilangan data (*lossless compression*) yang bekerja dengan menguraikan data menjadi ukuran lebih kecil dan menyusunnya kembali secara utuh ke bentuk semula. Kompresi dengan kehilangan data (*lossy compression*), yaitu teknik yang menghilangkan sebagian bit informasi setelah proses kompresi dilakukan. (Silitonga *et al.*, 2019; Hidayat *et al.*, 2019).

2.2.3. Format Berkas Data Suara.

Format berkas data suara terdiri atas WAV, MP3, dan FLAC. WAV atau *Wave File* merupakan format berkas suara yang dikembangkan oleh Microsoft dan IBM. *Wave file* disimpan menggunakan PCM (*Pulse Code Modulation*) sehingga termasuk berkas suara yang tidak terkompres sehingga menjadi format ideal untuk berkas suara rekaman professional. (Silitonga et al, 2019; Hall, 2024). MP3 (MPEG Audio Layer-3) merupakan contoh dari kompresi dengan kehilangan data (*Lossy Compression*) yang dikembangkan oleh Fraunhofer. Tujuan utama pengembangannya adalah untuk mengurangi ukuran berkas suara dengan tetap menjaga kualitas yang dapat diterima oleh pendengar. Format MP3 memiliki rasio kompresi antara 10:1 hingga 12:1, sehingga berkas suara berukuran 50 MB dalam format CD dapat dikompres menjadi sekitar 4–5 MB, dengan bitrate antara 128 kbps hingga 192 kbps (Hall, 2024). Selain MP3 terdapat FLAC (*Free Lossless Audio Codec*),

yang diperkenalkan oleh Josh Coalson pada tahun 2001, merupakan contoh dari kompresi tanpa kehilangan data (*Lossless Compression*). Format FLAC mampu mengurangi ukuran berkas suara sebesar 30% hingga 50% tanpa menurunkan kualitas audio aslinya. Sebagai ilustrasi, berkas suara dalam format CD dapat dikompres menjadi berukuran sekitar 700 hingga 1.000 kbps, dengan kualitas yang identik dengan sumber aslinya (Hall, 2024).

2.2.4. Automatic Speech Recognition.

Automatic Speech Recognition (ASR) merupakan teknologi yang memperbolehkan komputer mengambil file audio atau rekaman suara langsung dari pengeras suara dan mengubahnya menjadi teks. Teknologi ASR secara sederhana mentranskripsikan bahasa lisan menjadi skrip atau teks, selanjutnya melakukan pengenalan kata yang diucapkan dan menggunakannya sebagai data untuk diproses pada mesin selanjutnya. (Malik et.al 2021). Proses pengenalan ucapan manusia dimulai ketika sistem merekam ucapan manusia dan diubah menjadi gelombang ucapan. Selanjutnya, sistem mengubah sinyal ucapan menjadi urutan vector yang diukur sepanjang durasi sinyal ucapan. Kemudian, mendekode untuk menghasilkan urutan kalimat yang sesuai. (Deshmukh, A. M, 2020).

Automactic Speech Recognition (ASR) memiliki tugas untuk memetakan bentuk gelombang akustik ke urutan kata yang sesuai. Arsitektur ASR terdiri atas interaksi tiga komponen inti. Komponen pertama adalah Model Akustik yang bertanggung jawab untuk memproses sinyal audio masukan dan mengubahnya menjadi urutan vektor fitur akustik yang terkompresi. Komponen kedua adalah

Leksikon yang mendefinisikan himpunan unit yang pada umumnya berisi token subkata. Token subkata menjadi unit dasar yang dapat diprediksi sistem. Komponen ketiga adalah Model Bahasa yang memetakan probabilitas pada urutan kata. (Jurafsky et al, 2025).

2.2.5. Stratified Sampling.

Pada *Stratified Sampling* populasi dibagi ke dalam berbagai kelompok berbeda yang disebut strata berdasarkan kategori atau tingkat tertentu. Kemudian sampel acak diambil dari setiap starta yang mewakili kesuluruhan rentang strata. (Ahmed, 2024).

Formula *Stratified Sampling* dapat dilihat pada 2.1.

$$n_h = \frac{N_h}{N} \times n \qquad \textbf{(2.1)}$$

Keterangan :

n_h	:	Ukuran sampel untuk strata h.
N_h	:	Ukuran populasi untuk starta h.
N	:	Ukuran total populasi.
n	:	Ukuran total sampel.

2.2.6. Preprocessing.

Preprocessing merupakan proses untuk mempersiapkan gelombang suara untuk tahapan ekstraksi fitur. Proses awal dari *preprocessing* adalah persiapan konversi analog ke digital dengan 2 langkah yakni *sampling* dan *quantization*. *Sampling* merupakan tahapan pembuatan

sample rate dengan menempatkan sejumlah titik dari gelombang audio untuk mendapatkan representasi diskrit dalam satuan waktu. *Sample rate* merupakan jumlah sampel dalam satuan waktu dan dinyatakan dalam *Hertz (Hz)*. *Quantization* merupakan proses pemetaan nilai amplitudo dari setiap *sample rate* ke nilai dengan ketelitian bit. (Jurafsky et al., 2025). Tahapan *preprocessing* selanjutnya yakni *Voice Activity Detection* untuk mempertahankan hanya bagian ucapan dari gelombang ucapan. (Labied et al, 2022).

2.2.7. Data Splitting.

Data splitting merupakan pendekatan yang digunakan dalam proses validasi model pada pembelajaran mesin dengan memisahkan data ke dalam data pelatihan dan data pengujian (Joseph, 2022). Xu et al. (2018) menyampaikan bahwa fenomena overfitting dapat terjadi ketika model divalidasi menggunakan jumlah data pengujian yang terlalu sedikit. Oleh sebab itu, pembagian data yang akurat menjadi sangat penting. Data dibagi menjadi dua kelompok utama, yaitu data pelatihan dan data validasi. Data pelatihan dipecah menjadi dua sub bagian data yang digunakan untuk melatih model dan data yang digunakan untuk melakukan optimasi parameter model. Sementara itu, data validasi tidak dilibatkan dalam proses pelatihan, sehingga data tersebut dimanfaatkan untuk menilai akurasi model.

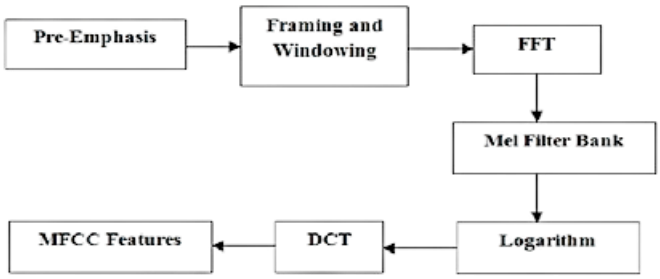
2.2.8. Feature Extraction.

Pada *Automatic Speech Recognition* mengubah gelombang ucapan mentah menjadi rangkai vektor akustik

melalui proses *feature extraction*. *Feature Extraction* merupakan *metode* untuk menghapus informasi yang tidak relevan dan mengekstrak informasi penting dari gelombang ucapan yang akan digunakan untuk tahap pembuatan model. Terdapat dua pendekatan dalam tahap *feature extraction*, yaitu pendekatan jaringan konvolusi yang melakukan pemrosesan langsung dari sinyal mentah , dan pendekatan *log mel spectrum* dan MFCC yang melakukan representasi tingkat lanjut. (Jurafsky et al., 2025).

Mel Frequency Capstral Coefficient (MFCC) merupakan metode ekstraksi fitur yang digunakan pada sistem pengenalan dan verifikasi pembicara. Metode MFCC berbariss sistem pendengaran manusia yang tidak dapat mendengar frekuensi diatas 1 KHz. (Jurafsky et al., 2025). Metode Mel-Frequency Cepstral Coefficients (MFCC) memiliki beragam kegunaan dalam analisis akustik, terutama dalam bidang pengenalan wicara otomatis, pengenalan emosi, identifikasi bahasa maupun dialek, pengenalan pembicara. klasifikasi gender melalui panggilan telepon, dan sistem deteksi suara sintetis. (Abdul et.al, 2017).

Gambar 2.1 merupakan penggambaran alur penerapan ekstraksi fitur menggunakan metode MFCC.



Gambar 2.1 Ekstraksi fitur menggunakan metode MFCC.

Alur ekstraksi fitur menggunakan metode MFCC terdiri atas *Pre-Emphasis, Frame Blocking, Windowing, Fast Fourier Transformers (FFT), Mel Filter Bank, Logarithm, Discrete Cosine Transformers, deltas and double deltas.*

1. *Pre-Emphasis*.

   Sinyal yang telah ditangkap y(n), dilewatkan melalui filter *high-pass,* untuk meningkatkan amplitude sinyal frekuensi tinggi dan mengompresi proses produksi ucapan manusia yang cenderung melemahkan frekuensi tinggi.

   Formula *Pre-Emphasis* dapat dilhat pada 2.2.

   $$\boldsymbol{X(n) = y(n) - a \times y\,(n-1)} \qquad \textbf{(2.2)}$$

   Keterangan :

X(n)	:	Sinyal keluaran.
y(n)	:	Sinyal ucapan asli pada sampel ke-n.
a	:	Koefisien *pre-emphasis* (antara 0.85 hingga 1)

2. *Frame Blocking.*

   Sinyal masukan dibagi menjadi sejumlah bingkai jangka pendek yang disusun secara tumpang tindih agar mencapai kontinuitas sinyal.

3. *Windowing.*

   Semua bingkai dikalikan dengan fungsi jendela agar memastikan kontinuitas pada titik akhir bingkai. Fungsi jendela yang digunakan adalah Hamming.

   Formula Hamming dapat dilihat pada 2.3.

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right) \quad (2.3)$$

Keterangan :

W(n)	:	Nilai window pada posisi ke-n.
n	:	Sampel.
N	:	Jumlah total sampel dalam satu frame.
Cos	:	Fungsi kosinus.

4. *Fast Fourier Transformers (FFT).*
   Melakukan kalkulasi respons frekuensi magnitudo untuk setiap bingkai.

5. *Mel Filter Bank.*
   Melakukan kalkulasi antara respons frekuensi magnutudo setiap bingkai dengan serangkaian filter *bandpass* segitiga untuk memperoleh energi logaritma yang sesuai dari setiap filter.
   Formula frekuensi mel (*f)* dapat dilihat pada 2.4

$$Mel(f) = 1127\, In\left(1 + \frac{f}{700}\right) \quad (2.4)$$

Keterangan :

Mel(f)	:	Frekuensi pada skala mel.
1127	:	Faktor skala.
In	:	Logaritma natual.

$\left(1 + \frac{f}{700}\right)$	:	Koreksi logaritmik awal.
$\frac{f}{700}$	:	Normalisasi frekuensi.

6. *Logarithm.*

   Melakukan kalkulasi logaritma spektrum magnitudo karena pendengaran manusia tidak bisa mendengar suara dalam skala linear.

7. *Discrete Cosine Transformers.*

   Melakukan kalkulasi DCT untuk mendekorelasi energi bank filter yang tumpang tindih. DCT diterapkan pada nilai log energi yang dihitung dari filter segitigas untuk merepresentasikan data sinyal dalam domain waktu.

   Formula *Discrete Cosine Transformers (DCT)* dapat dilihat pada 2.5.

$$\boldsymbol{C_n = \sum_{k-1}^{k} \log(sk)\, cos\left(\frac{\pi n}{K}(k - \frac{1}{2})\right)} \quad \textbf{(2.5)}$$

Keterangan :

C_n	:	Koefissien MFCC ke-n
$\sum_{k-1}^{k}$	:	Penjumlahan dari semua filter Mel.
k	:	Indeks filter Mel.
K	:	Total filter Mel.
$\log(sk)$	:	Log-energi Mel.
$(k - 0.5)$	:	Kofeksi fasa.
n	:	Indeks koefisien

8. *Deltas and double deltas.*

   Sinyal ucapan antar bingkai bersifat tidak konstan, sehingga perlu melakukan kalkulasi fitur terkait perubahan waktu.

   Formula koefisien delta dapat dilihat pada 2.6.

$$\boldsymbol{\delta(t) = c(t+1) - c(t)} \qquad \textbf{(2.6)}$$

Keterangan :

$\boldsymbol{\delta(t)}$	**:**	Nilai perubahan
$\boldsymbol{c(t+1)}$	**:**	Koefisien MFCC pada bingkai waktu selanjutnya.
$\boldsymbol{c(t)}$	**:**	Koefisien MFCC pada bingkai waktu ke-t.

2.2.9. Acoustic Model.

Model akustik merupakan model yang didesain untuk menganalisis rekaman wicara dan menghasikan representasi statistik dari bunyi yang membentuk kata. Salah satu model akustik yang popular adalah *Gaussian Mixture Model* yang dikombinasikan dengan *Hidden Markov Model* sebagai *decoder*. GMM merupakan model probabilistik yang mengombinasikan parameter yang telah dilatih dengan unsur keacakan, sedangkan HMM menghitung transisi antar bunyi. Kombinasi dari GMM-HMM berfungsi untuk meningkatkan akurasi dan menjadi fondasi dari ASR klasik. Akibat perkembangan teknologi, model DNN dikenalkan sebagai model akustik dalam pengenalan wicara otomatis. Model DNN berfungsi untuk menghitung kemungkinan posterior dari *state* HMM yang menggantikan fungsi GMM.(Fatehi et al., 2025).

Model akustik dapat dinyatakan sebagai P(X | S) yang menunjukan probabilitas untuk observasi sinyal wicara X berdasarkan urutan *hidden state* S. *Hidden State* merupakan representasi fonetik yang merupakan bagain dari fonem.Pada tahap awal pemrosesan model akustik, setiap sinyal wicara ditransformasikan menjadi serangkaian vektor fitur per bingkai waktu. Tahap selanjutnya adalah menggunakan aturan probabilitas dan asumsi independensi observasi HMM. Sehingga, probabilitas P(X | S) merupakan hasil perkalian probabilitas setiap bingkai terhadap *state*. (Wang et, al., 2019)

Formula dari probabilitas P(X | S) dapat dilihat pada 2.7.

$$p(X|S) = \prod_{t=1}^{T} p(x_t|x_1, \dots, x_{t-1}, S)$$
$$\approx \prod_{t=1}^{T} p(x_t|s_t) \propto \prod_{t=1}^{T} \frac{p(s_t|\, x_t)}{p(s_t)} \quad \textbf{(2.7)}$$

Keterangan :

p(X|S)	**:**	Probabilitas dari observasi X terhadap *hidden state* S.
∏t=1Tp(xt|x1,...,xt−1,S)	**:**	Bentuk umum asumsi independensi.
∏t=1Tp(xt|st)	**:**	Asumsi independensi
∏t=1Tp(st)p(st|x)	**:**	Bentik basis posterior.

2.2.10. Gaussian Mixture Model.

Fenomena statistik dalam data memiliki pola sebaran yang berpusat disuatu nilai tengah yang secara matematis menggunakan distriusi Gaussian. Distribusi Gaussian merupakan probabilitas kemunculan data tertinggi berada di titik retata dan berangsur menurun ketika menjauh dari titik rerata. Pada sistem pengenalan wicara otomatis diperlukan lebih satu distribusi karena variasi fonetik. Pemodelan dari kondisi sistem pengenalan wicara otomatis menggunakan GMM yang menggabungkan beberapa distribusi Gaussian. (GM et al., 2020).

Formula Gaussian *Mixture* dapat dilihat pada 2.8.

$$P(x \mid \vartheta) = \sum_{k=1}^{N} w_k b(x \mid \mu_k, \rho_k) \quad \textbf{(2.8)}$$

Keterangan :

x	:	Vektor fitur.
ϑ	:	Parameter GMM.
N	:	Jumlah komponen Gaussian.
w_k	:	Bobot komponen ke-k
$b(x \mid \mu_k, \rho_k)$	:	Fungsi densitas Gaussian Multivariat.

2.2.11. Deep Neural Network.

Deep Neural Network (DNN) merupakan bagian dari jaringan saraf tiruan *feed-forward* yang terdiri dari sejumlah lapisan tersembunyi diantara lapisan *input* dan lapisan *output*. DNN mampu mempelajari representasi fitur *input* melalui lapisan tersembunyi dan efektif dalam menangani variasi pada koefisien *input* akustik. (Singh et

al.,2018). DNN meningkatkan kapasitas representasi model melalui jumlah parameter jaringan yang besar. Pada eksperimen tahun 2011, ditunjukan bahwa DNN memberikan peningkatan kinerja pada dataset pengenalan wicara berbasis *monophone state* yang kemdian diterapkan pada *triphone state*. Pada sistem ASR, *Deep Neural Network* (DNN) memperkirakan probabilitas posterior p (y | x) untuk setiap state fonetik. Penggunaan aturan bayes terhadap keluaran posterior perlu digunakan untuk menjamin perkiraan posterior dari DNN dapat digunakan oleh HMM. (Maas et al., 2015)

Formula dari aturan bayes dapat dilihat pada 2.9.

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)} \quad \textbf{(2.9)}$$

Keterangan :

$P(x \mid y)$	:	Kemungkinan akustik
$p(y \mid x)$	:	Posterior DNN
$p(y)$	:	Frekuensi kemunculan *state.*
$p(x)$	:	Nilai normalisasi.

Secara matematis, *Delay Neural Network* (DNN) memiliki fungsi *logistic* yang digunakan untuk memetakan informasi unit dari lapisan tersembunyi sebelumnya ke nilai skalar.

Formula dari fungsi logistic untuk memetakan informasi unit ke nilai skalar dapat dilihat pada 2.10.

$$y_j = logistic(x_j) = \frac{1}{1 + e^{-x_j}}, x_j = b_j + \sum_i y_i w_{ij,}$$

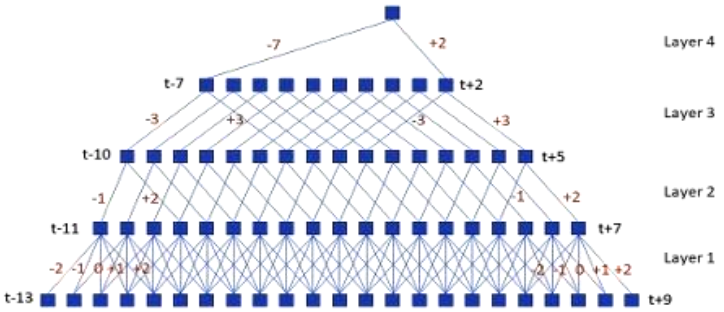
(2.10)

Keterangan :

y_j	:	Nilai keluaran dari neuron ke-j.
$logistic(x_j)$	:	Fungsi aktivasi logistic.
$\frac{1}{1+e^{-x_j}}$	:	Mengubah input linear menjadi probabilitas ternomalisasi.
x_j	:	Nilai masukan total ke neuron j.
b_j	:	Bias neuron j.
$\sum_i y_i w_{ij,}$	:	Hasil kalkulasi antara nilai keluaran neuron sebelumnya dengan bobot koneksi ke neuron j.

2.2.12. Time Delay Neural Network.

Time Delay Neural Network (TDNN), diperkenalkan oleh Alex Waibel dan tim peneliti pada tahun 1987 pada penelitian pengenalan wicara. Melalui penelitian pengenalan wicara, Alex dan tim peneliti menunjukan proses TDNN dapat mengidentifikasi fitur temporal dalam data wicara. TDNN merupakan turunan dari jaringan syaraf tiruan yang mengenali pola temporal pada data dengan kemampuan mengakses informasi dari waktu sebelumnya. Memasukan lapisan tertunda pada TDNN membuat model mampu mengenali pola sepanjang waktu. Struktur dari TDNN untuk pengenalan wicara dirancang untuk melakukan pengenalan pola yang memiliki dependensi waktu. (Gunarso et al., 2024). Gambar 2.2 menunjukan contoh struktur TDNN yang terdiri atas 4 lapisan.



Gambar 2.2 struktur TDNN dengan 4 lapisan.

Lapisan awal dalam Struktur TDNN adalah lapisan *input* yang menerima fitur akustik dari sinyal wicara. Fitur akustik berupa *Mel-Frequency Cepstral Coefficients* (MFCC) yang menggambarkan aspek penting dari spektrum suara. Setelah input, terdapat lapisan waktu-tunda yang terdiri atas satu atau beberapa lapisan. Neuron pada lapisan waktu-tunda terhubung dengan *input* saat ini dan *input* dari waktu sebelumnya. Karena setiap neuron pada lapisan waktu-tunda terhubung dengan beberapa titik waktu sebelumnya yang memungkinkan jaringan untuk "melihat ke belakang" untuk mempelajari dependensi temporal. (Gunarso et al., 2024).

Setelah lapisan waktu-tunda, TDNN memiliki satu atau lebih lapisan tersembunti untuk mempelajari transformasi non-linear pada data dan membantu mengindentifikasi pola kompleks dalam wicara. Lapisan terakhir dalam struktur TDNN adalah lapisan *output* atau lapisan *softmax* untuk mengklasifikasikan *input* ke dalam kategori, seperti fonem, kata, atau frasa. Lapisan *output* memberikan probabilitas *input* terhadap kategori akhir yang memudahkan identifikasi dan transkripsi wicara. (Gunarso et al., 2024).

2.2.13. ReLU.

Rectified Linear Units (ReLU) merupakan fungsi non-linear yang mengaktifkan neuron ketika nilai keluaran hasil transformasi linear lebih besar dari 0. Formula dari ReLU dapat dilihat pada 2.11.

$$f(x) = max(0, x) \tag{2.11}$$

Keterangan :

X	:	Nilai masukan
F(x)	:	Hasil keluaran fungsi ReLU berupa nilai dalam x atau 0.

2.2.14. Softmax.

Softmax merupakan fungsi aktivasi softmax yang menghitung probabilitas untuk menentukan klasifikasi multi kelas dengan output kelas yang memiliki nilai probabilitas tertinggi. (Mustakim et al, 2021) Formula dari softmax dapat dilihat pada 2.12.

$$\mathrm{f(Xi)} = \frac{\mathrm{Exp(Xi)}}{\sum_{j=0}^{k} \mathrm{Exp(Xj)}} \tag{2.12}$$

Keterangan :

Xi	:	Nilai input ke-i.
Exp(Xi)	:	Eksponensial dari Xi.
$\sum_{j=0}^{k} \mathrm{Exp(Xj)}$	:	Penjumlahan seluruh nilai eksponensial dari input Xj.

2.2.15. Lexicon.

Lexicon atau kamus pelafalan merupakan jembatan antara model akustik yang menggunakan fonem dan model

bahasa yang menggunakan kata. Lexicon digunakan untuk mentranslasikan kata atau kalimat menjadi urutan fonem dengan memetakan setiap kata dalam model bahasa ke urutan fonem. (Gunarson, 2019). Kamus pelafalan dapat dirumuskan sebagai p(S|L) yang mencakup pemetaan dari pelafalan, fonem atau dari fonem ke *triphone,* dan pemetaan hubungan perhitungan probabilitas. (Wang et al., 2019)

2.2.16. Hidden Markov model (HMM)

Hidden Markov Model (HMM) merupakan proses stokastik yang terdiri dari rantai Markov tersembunyi dengan sejumlah state terbatas yang menghasilkan observasi melalui fungsi acak tertentu. Pada setiap langkah waktu, sistem berada pada sebuah state, menghasilkan observasi, dan berpindah ke state lain berdasarkan probabilitas transisi. HMM memiliki tiga komponen probabilistic utama, yakni matriks transisi yang berisi probabilitas transisi dari rantai Markov berpindah dari state saat ini (i) ke state selanjutnya state j. Matriks observasi yang merupakan fungsi acak yang diasosiasikan dengan state j atau biasa disebutu campuran terbatas dari distribusi Gaussian. Komponen terakhir dari HMM adalah Distribusi state awal. (Mansouri et al.,2021).

Formula matriks distribusi probabilitas transisi dapat diliat pada 2.13.

$$a_{ij} = P(q_t + 1 \ = \ \text{j} | q_t = i) \quad \textbf{(2.13)}$$

Keterangan :

a_{ij}	:	Probabilitas transisi
q_t	:	*State* pada waktu t

Formula matriks distribusi probabilitas transisi dapat diliat pada 2.14.

$$b_j(o) = \sum_{k=1}^{M} c_{jk} G\left(o; \mu_{jk}, U_{jk}\right), 1 \leq j \leq N; \quad \textbf{(2.14)}$$

Keterangan :

0	:	Vektor observasi.
c_{jk}	:	Koefisien campuran
G	:	Distrubusi Gaussian dengan vektor rata-rata.
μ_{jk}	:	Vektor rata-rata.
U_{jk}	:	Matriks kovarian.

Formula matriks distribusi probabilitas transisi dapat diliat pada 2.15.

$$\pi_i = P(q_1 = i), 1 \leq i \leq N. \quad \textbf{(2.15)}$$

Keterangan :

π_i	**:**	Probabilitas urutan dari *state* i.
q_1	**:**	*State* pada waktu awal.
N	**:**	Jumlah total *state.*

2.2.17. Language Model

Model bahasa digunakan untuk membantu mengubah rangkaian unit fonetik menjadi kata dan kalimat yang bermakna dengan mempertimbangkan tata bahasa. Pada Automatic Speech Rexognition, model bahasa memiliki peran untuk meningkatkan akurasi dengan memastikan proses decoding tetap konsisten secara tata bahasa. (Fatehi et al,.2025). Model bahasa dapat dirumuskan

sebagai p(L) dilatih menggunakan sejumlah data dengan menerapkan hipotesis Markov orde m-1 untuk menghasilkan model bahasa m-gram. (Wang et al., 2019) .

Formula probabilitas dari model bahasa dapat dilihat pada 2.16.

$$p(L) = \prod_{n=1}^{N} p(l_n \mid l_1, \dots, l_{n-1}) \quad \textbf{(2.16)}$$
$$= \prod_{n=1}^{N} p(l_n \mid l_{n-m-1}, \dots, l_{n-1})$$

Keterangan :

L	:	Urutan fonem dalam kalimat
l_n	:	Token ke-n
$p(l_n \mid l_1, \dots, l_{n-1})$	:	Probabilitas penuh
$p(l_n \mid l_{n-m-1}, \dots, l_{n-1})$	:	Aproksimasi n-gram

2.2.18. N-Gram.

N-Gram merupakan model probabilistic yang mengestimasikan probabilitas urutan kata yang diberikan berdasarkan n-1 kata sebelumnya. N pada n gram menunjukan urutan kata yang mana 2-gram atau bigram merupakan urutan dari dua kata sedangkan 3-gram atau trigram merupakan urutan dari tiga kata. Probabilitas urutan kata dinotasikan sebagai *P(w|h)* yang mana w merupakan kata yang akan diestimasikan probilitasnya sedangkan h adalah rangkaian kata sebelumnya. Perhitungan probabilitas dari keseluruhan rangkaian dilakukan menggunakan aturan rangkaian probabilitas. (Jufafsky et al, 2025).

Formula aturan rangkaian probabilitas dapat dilihat pada 2.17

$$p(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \quad (2.17)$$

$$= \prod_{k=1}^{N} P(w_k|w_{1:k-1})$$

Keterangan :

$p(w_{1:n})$	**:**	Probabilitas dari kata dari w1 hingga wn
$P(w_1)$	**:**	Probabilitas kata pertama.
$P(w_n\|w_{1:n-1})$	**:**	Probabilitas kata ke-n dengan syarat kata sebelumnya telah diketahui.
$\prod_{k=1}^{N}$	**:**	Perkalian berulang dari indeks k =1 hingga k = N

Penerapan probabilitas aturan rangkaian pada 2. Dapat diubah menjadi perhitungan mengunakan format kondisional dari kata selanjutnya menggunakan asumsi Markov.

Formula asumsi Markov dapat dilihat pada 2.18.

$$p(w_{1:n}) \approx \prod_{k=1}^{N} P(w_k|w_{1:k-1}) \quad (2.18)$$

Keterangan :

$p(w_{1:n})$	**:**	Probabilitas dari kata dari w1 hingga wn
$\approx$	**:**	Pendekatan.

$\prod_{k=1}^{N}$	:	Perkalian berulang dari indeks k =1 hingga k = N
$P(w_k \mid w_{1:k-1})$		Probabilitas kata ke-k bersyarat seluruh kata sebelumnya diketahui.

Asumsi Markov menentukan probabilitas rangkaian kata dan diperlukan estimasi numerik dari rangkaian kata. Estimasi numerik untuk rangkaian kata disebut *Maximum Likelihood Estimation*. MLE memberikan estimasi numerik untuk menentukan urutan kata yang paling sesuai berdasarkan hasil estimasi.

Formula MLE n-gram secara umum dapat dilihat pada 2.19.

$$P(w_n \mid w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1} w_n)}{C(w_{n-N+1:n-1})} \quad (2.19)$$

Keterangan :

$P(w_n \mid w_{n-N+1:n-1})$	:	Probabilitas bersyarat n-gram
N	:	Ukuran n-gram.
C	:	Jumlah kemunculan
$C(w_{n-N+1:n-1} w_n)$	:	Jumlah kemunculan frasa
$C(w_{n-N+1:n-1})$	:	Jumlah kemunculan konteks

Kalkulasi estimasi menggunakan MLE dapat menghasilkan nilai 0 untuk setiap rangkaian probabilitas yang jarang muncul sehingga memicu bias data. Maka, metode smoothing menggunakan Kneser-Ney diterapkan untuk menangani masalah bias data. Kneser-Ney memberikan peluang sebuah kata muncul dalam rangkaian yang berbeda berdasarkan jumlah rangkaian unik sebelumnya.

Formula dari Kneser-Ney dapat dilihat pada 2.20.

$$P(w_i|w_{i-n+1:i-1}) = \frac{\max(C_{KN}(w_{i-n+1:i}) - d, 0)}{\sum_v C_{KN}(w_{i-n+1:i-1}v)} + \lambda(w_{i-n+1:i-1})P_{KN}(w_i|w_{i-n+2:i-1})$$

(2.20)

Keterangan :

$P(w_i\|w_{i-n+1:i-1})$	:	Probabilitas kemunculan kata wi setelah konteks n-1 kata sebelumnya.
w_i	:	Kata yang diamati probabilitasnya.
$w_{i-n+1:i-1}$	:	Urutan kata dari i-n+1 hingga i-1.
$w_{i-n+1:i}$	:	Urutan konteks ditambah rangakian sepanjang n kata.
$C_{KN}(w_{i-n+1:i})$	:	Jumlah kemunculan n-gram dalam korpus
d	:	Angka pengurangan.
$\max(C_{KN}(w_{i-n+1:i}) - d, 0)$	:	Mengurangai frekuensi dengan d dan mencegah probabilitas negatif.
$\sum_v C_{KN}(w_{i-n+1:i-1}v)$	:	Normalisasi penyebut dengan hasil jumlah seluruh n-gram dengan konteks serupa.
v	:	Kata yang termasuk pada konteks.
$\lambda(w_{i-n+1:i-1})$	:	Backoff weight
$P_{KN}(w_i\|w_{i-n+2:i-1})$	:	Probabilitas backoff

2.2.19. Viterbi.

Algoritma Viterbi menerapkan pemrograman dinamis untuk menyelesaikan masalah decoding pada HMM. Proses decoding merujuk pada proses rekursif dari dua variabel kunci, yakni δt(j) sebagai probabilitas tertinggi pada urutan state yang sepanjang waktu awal hingga akhir dari state j pada waktu t, dan ψt(j) yang menyimpan indeks dari state sebelumya dengan nilai tertinggi untuk δt(j). (Deng et al, 2025) Fromula dari variabel kunci dapat dilihat pada 2.21.

$$\delta 0(j) = \pi j \cdot Bj\,(x0)$$
$$\psi 0(j) = 0 \qquad (2.21)$$

Keterangan :

πj	:	Probabilitas state awal
Bj (x0)	:	Emission Probability observasi pertama.
ψ0(j)	:	Initial backpointer.
δ0(j)	:	Skor Viterbi pada waktu ke-0 pada state j.

Initial backpointer pada tahap inisialisasi memiliki nilai 0 dikarenakan tidak terdapat state sebelum state pertama. Kedua variabel kunci bekerja secara rekursif dengan formula rekursif Viterbi dapat dilihat pada 2.22.

$$\delta t(j) = max_{1 \le i \le K}\,[\delta t - 1(i) \cdot Aij] \cdot Bj(xt) \qquad (2.22)$$

Keterangan :

$\delta t(j)$	:	Skor Viterbi pada waktu t
max_i	:	Operator maximum.
$\delta t - 1(i)$	:	Skor Viterbi pada waktu t-1 untuk state i.

Aij	:	Probabilitas transisi perpindahan state i ke state j.
$Bj(xt)$	:	Probabilitas Emisi state j menghasilkan observasi x

Pada proses penentuan urutan state terbaik algoritma viterbi memerlukan pemilihan state sebelumnya melalui formula backpropagation yang dapat dilihat pada 2.23.

$$\psi t(j) = argmax\,[\delta t - 1(i) \cdot Aij] \qquad (2.23)$$

Keterangan :

$\psi t(j)$	:	Backpointer state j pada waktu t
$argmax$	:	Operator argument maksimum.
$\delta t - 1(i)$	:	Skor Viterbi pada waktu t-1 untuk state i.
Aij	:	Probabilitas transisi perpindahan state i ke state j.

2.2.20. Hyperparameter Tunning.

Hyperparameter Tunning merupakan proses menemukan kombinasi *hyperparameter* optimal yang memberikan performa terbaik pada sebuah model pembelajaran mesin. Penentuan *hyperparameter* penting dikarenakan menentukan proses pembelajaran dan struktur model seperti *learning rate* dan *jumlah layer* dalam neural network. Pemilihan kombinasi yang tepayt dapat meningkatkan akurasi, mempercepat konvergensi, dan membantu menyeimbangkan bias dan varians sehingga mampu melakukan generalisasi pada data baru. Pada sisi yang lain, pemilihan *hyperparameter* yang keliru dapat menyebabkan model mengalami underfitting, overfitting, dan pelatiahn yang tidak efisien. (Llemabayo, et al 2024).

2.2.21. Word Error Rate (WER).

Word Error Rate merupakan metrik evaluasi ketepatan dari sebuah model *Automatic Speech Recognition (ASR)* bukan pada level fonem melainkan level kata. (Malik et. al, 2021). Word Error Rate berdasar pada perbedaan antara rangkaian kata yang dihasilkan oleh sistem pengenal atau hipotesis dengan transkripsi yang dianggap benar. Perhitungan Word Error Rate dilakukan dengan menghitung jarak penyuntingan minimum dengan satuan kata antara hipotesis dan referensi. Hasil dari perhitungan jarak penyuntingan minimum adalah jumlah minimum operasi yang terdiri dari subsitusi kata, penyisipan kata, dan penghapusan kata yang digunakan untuk memetakan hipotesis dengan referensi. (Jurafsky et al., 2024).

Formula Perhitungan *Word Error Rate* dapat dilihat pada 2.24:

$$\boldsymbol{WER = \frac{S + D + I}{N}} \qquad \textbf{(2.24)}$$

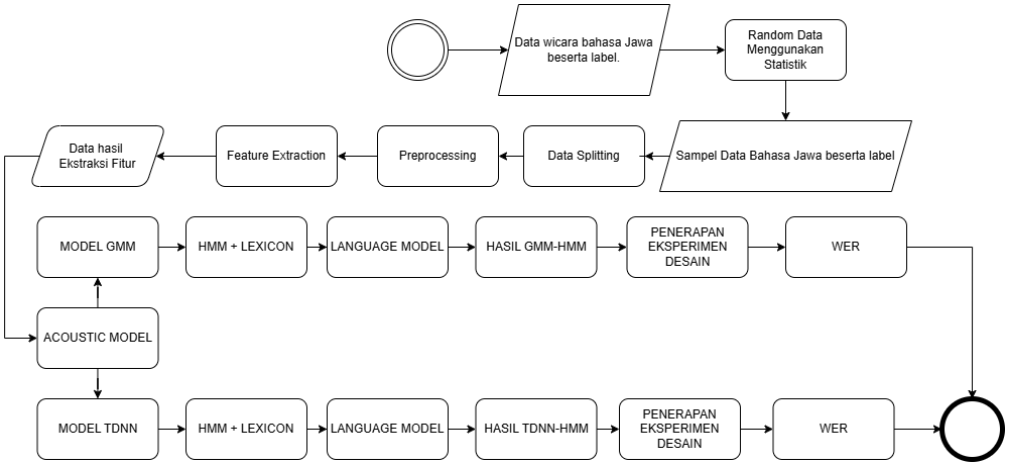
Keterangan :

S : Jumlah Substitusi kata dalam teks *output* dibandingkan dengan teks acuan.
D : Jumlah penghapus kata.
I : Jumlah penyisipan kata.
N : Jumlah total kata dalam teks acuan.

BAB III METODE PENELITIAN

3.1. Alur Proses Penelitian.

Penelitian perbandingan metode klasik GMM dan TDNN dalam data wicara bahasa Jawa dengan HMM sebagai *decoder* untuk kedua algoritma. Metode GMM dan TDNN dengan HMM sebagai *decoder* diterapkan dan dibandingkan dengan tujuan memperlihatkan kinerja metode klasik GMM dan metode *deep learning* dalam pengenalan wicara menggunakan data bahasa Jawa. Tahap penelitian dapat dilihat pada gambar 3.1.

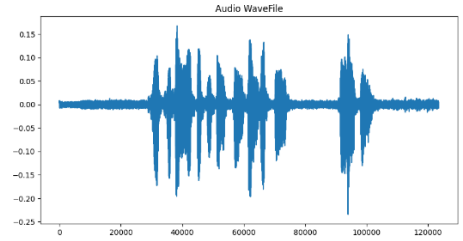
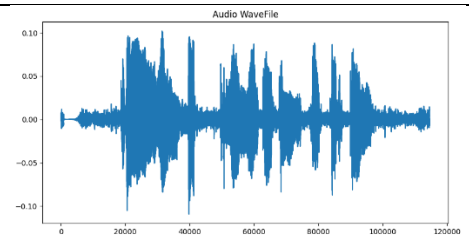


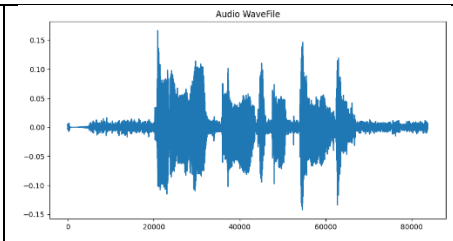
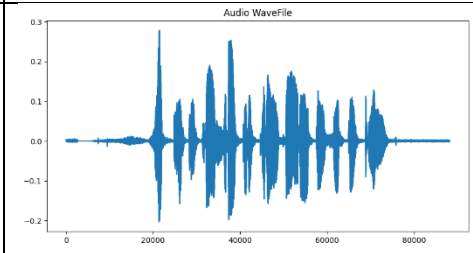
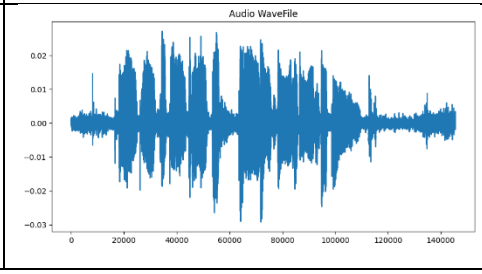
Gambar 3.1 Alur Penelitian

Tahapan alur penelitian terdiri atas persiapan data, pengambilan fitur dari data, penerapan model, penerapan eksperimen desain, dan pengukuran menggunakan metrik evaluasi WER. Tahap persiapan data merupakan tahap mempersiapkan data untuk dapat diproses melalui pengambilan fitur data yang terdiri atas pengambilan sampel data menggunakan Stratified *Sampling, data splitting* dan *preprocessing*. Tahap pengambilan fitur terdiri atas *pre-emphasis, framing and windowing, FFT, mel filter bank, logarithm, DCT,* dan *MFCC.* Tahap penerapan model terdiri atas penerapan model akustik GMM-HMM dan TDNN, lexicon, dan penerapan model bahasa. Tahapan akhir adalah penerapan eksperimen desain dan pengkuran dengan metwrik evaluasi Word Error Rate. Proses dari alur penelitian akan dijabarkan lebih lanjut pada sub bab 3.2 hingga 3.4.

3.2. Data.

Data yang digunakan pada penelitian mengenai perbandingan metode akustik GMM dan TDNN yakni data wicara berisi potongan percakapan dalam bahasa Jawa. Data wicara diambil dari website open slr yang menyediakan berbagai jenis data wicara dan bahasa. Salah satu diantaranya yakni data wicara dalam bahasa Jawa. Data bahasa jawa terdiri atas 185.076 rekaman suara dalam format *FLAC* yang dilengkapi label dan dipisahkan menjadi 16 paket data. Penelitian perbandingan kinerja algoritma GMM dan TDNN tidak menggunakan keseluruhan melainkan menggunakan 10% dari dataset. Pengambilan data yang akan digunakan untuk proses menggunakan metode statistik yakni *Stratified Sampling* dan menghasilkan 18.501 untuk data bahasa Jawa. Penerapan *Stratified Sampling* sebagai metode pengambilan data didasari dengan metode probabilistik data yang terdiri atas tingkatan sehingga seusai dataset yang tediri atas berbagai paket dan folder yang merupakan tingkatan. Data kemudian diproses pada tahapan yang akan dijelaskan pada sub bab 3.3 dan akan diukur menggunakan metrik evaluasi WER (*Word Error Rate)*. Contoh data dapat dilihat pada tabel 3.3.1.

ID	Capture Gelombang Wicara	Transkrip
005ea73367		Driji kang nutupi tabung banjur diculaké
005c3b287f		Chow Yun Fat lunga dhateng Aberdeen

007b178920		Aaron Kwok lunga dhateng Guangzhou
005c9911c7		Hadi dikontrak kerja karo TV Tokyo
002b338ccc		Kéwan punika paling primitif tinimbang jinis

Tabel 3.3.3.1 Contoh data gelombang wicara

3.3. Metode Penelitian.

3.3.1. *Stratified Sampling.*

Penerapan *stratified sampling* pada data bahasa jawa dan bahasa sunda yang masing masing terdiri atas 16 folder data yang terdiri dari 16 subfolder pada tiap folder tersebut. Kondisi data yang memiliki tingkatan yang membuat *stratified sampling* digunakan sebagai metode pengambilan sampel. 16 folder data dapat dilihat pada tabel 3.3.1.1.

16 FOLDER DATA	
0	11654
1	11494
2	11587
3	11574
4	11498
5	11531
6	11782
7	11750

8	11629
9	11344
A	11537
B	11591
C	11456
D	11707
E	11498
F	11444
TOTAL	185076

Tabel 3.3.1.1 Tabel 16 Paket Data

Tabel 3.3.1.1 merupakan tabel jumlah data dari 16 paket data yang akan melalui proses *sampling.* Setiap folder memiliki jumlah data suara yang hampir sama dengan total data 185.076 berkas berformat FLAC. Proses selanjutnya adalah mengetahui jumlah berkas data suara pada tiap subfolder dari tiap folder. Jumlah berkas data pada tiap subfolder dapat dilihat pada tabel 3.3.1.2.

16 PAKET DATA-0		16 PAKET DATA-1		16 PAKET DATA-2		16 PAKET DATA-3	
0	752	0	672	0	745	0	749
1	699	1	701	1	713	1	732
2	762	2	710	2	728	2	754
3	714	3	737	3	706	3	705
4	710	4	687	4	749	4	701
5	730	5	741	5	763	5	703
6	701	6	711	6	729	6	715
7	741	7	740	7	741	7	732
8	722	8	702	8	686	8	724
9	721	9	716	9	743	9	743
A	709	A	747	A	707	A	716
B	766	B	738	B	726	B	734
C	743	C	729	C	714	C	718
D	733	D	696	D	704	D	731
E	745	E	712	E	746	E	699
F	706	F	755	F	687	F	718
TOTAL	11654	TOTAL	11494	TOTAL	11587	TOTAL	11574

16 PAKET DATA-4	
0	694
1	764
2	748
3	702
4	676
5	727
6	707
7	695
8	685
9	710
A	779
B	725
C	717
D	725
E	720
F	724
TOTAL	11498

16 PAKET DATA-5	
0	717
1	730
2	706
3	740
4	713
5	725
6	739
7	666
8	758
9	709
A	683
B	716
C	738
D	739
E	734
F	718
TOTAL	11531

16 PAKET DATA-6	
0	700
1	724
2	737
3	744
4	789
5	696
6	729
7	713
8	736
9	722
A	753
B	721
C	752
D	774
E	749
F	743
TOTAL	11782

16 PAKET DATA-7	
0	737
1	705
2	751
3	736
4	759
5	753
6	719
7	713
8	724
9	735
A	776
B	736
C	744
D	785
E	689
F	688
TOTAL	11750

16 PAKET DATA-8	
0	735
1	722
2	694
3	761
4	724
5	732
6	787
7	739
8	737
9	720
A	701
B	685
C	738
D	715
E	714
F	725
TOTAL	11629

16 PAKET DATA-9	
0	670
1	680
2	688
3	720
4	690
5	724
6	741
7	719
8	711
9	699
A	731
B	721
C	660
D	695
E	748
F	747
TOTAL	11344

16 PAKET DATA-A	
0	764
1	686
2	719
3	740
4	727
5	671
6	725
7	697
8	763
9	746
A	747
B	679
C	721
D	679
E	738
F	735
TOTAL	11537

16 PAKET DATA-B	
0	748
1	745
2	705
3	758
4	728
5	683
6	760
7	711
8	707
9	768
A	757
B	654
C	734
D	716
E	698
F	719
TOTAL	11591

16 PAKET DATA-C	
0	721
1	703
2	641
3	751
4	755

16 PAKET DATA-D	
0	719
1	736
2	742
3	703
4	742

16 PAKET DATA-E	
0	719
1	703
2	709
3	708
4	740

16 PAKET DATA-F	
0	702
1	714
2	739
3	734
4	715

5	731	5	732	5	693	5	711
6	695	6	737	6	704	6	700
7	731	7	765	7	703	7	711
8	702	8	765	8	723	8	667
9	784	9	749	9	705	9	704
A	706	A	723	A	762	A	704
B	699	B	699	B	730	B	726
C	706	C	707	C	772	C	765
D	694	D	758	D	729	D	732
E	713	E	693	E	731	E	672
F	724	F	737	F	667	F	748
TOTAL	11456	TOTAL	11707	TOTAL	11498	TOTAL	11444

Tabel 3.3.1.2 Data Subfolder.

Setelah mengetahui jumlah detail pada setiap subfolder di tiap folder, maka langkah selanjutnya adalah menentukan jumlah total data yang akan digunakan, yakni 10% dari total data. Pengambilan sampling menerapkan formula *Stratified Sampling* yang tertera pada 3.3.1.3.

$$\boldsymbol{n_h = \frac{N_h}{N} \times n} \qquad \textbf{(3.3.1.3)}$$

$\boldsymbol{n_h}$ merupakan ukuran sampel, **Nh** merupakan jumlah data pada subfolder , **N** merupakan jumlah data total, dan **n** merupakan perkiraan sampel pada folder. Detail contoh *stratified sampling* dapat dilihat pada perhitungan sampel yang tertera pada 3.3.1.4.

$$\begin{aligned} \boldsymbol{n_h} &= \frac{752}{11654} \times (\,11654 \\ &\qquad \times 10\%) \\ \boldsymbol{n_h} &= 0{,}064527201 \times 1{,}165{,}4 \\ \boldsymbol{n_h} &= 75{,}2000000454 \\ \boldsymbol{n_h} &= 75 \end{aligned} \qquad \textbf{(3.3.1.4)}$$

Hasil *Stratified Sampling* bukan merupakan bilangan bulat, sehingga perlu pembulatan bilangan untuk memastikan jumlah data per subfolder. Detail hasil *Stratified Sampling* dapat dilihat pada Tabel 3.3.1.5.

16 PAKET DATA-0	
0	75
1	70
2	76
3	71
4	71
5	73
6	70
7	74
8	72
9	72
A	71
B	77
C	74
D	73
E	75
F	71
TOTAL	1165

16 PAKET DATA-1	
0	67
1	70
2	71
3	74
4	69
5	74
6	71
7	74
8	70
9	72
A	75
B	74
C	73
D	70
E	71
F	76
TOTAL	1151

16 PAKET DATA-2	
0	75
1	71
2	73
3	71
4	75
5	76
6	73
7	74
8	69
9	74
A	71
B	73
C	71
D	70
E	75
F	69
TOTAL	1160

16 PAKET DATA-3	
0	75
1	73
2	75
3	71
4	70
5	70
6	72
7	73
8	72
9	74
A	72
B	73
C	72
D	73
E	70
F	72
TOTAL	1157

16 PAKET DATA-4	
0	75
1	73
2	75
3	71
4	70
5	70
6	72
7	73
8	72
9	74
A	72
B	73
C	72
D	73
E	70
F	72
TOTAL	1157

16 PAKET DATA-5	
0	71
1	73
2	70
3	74
4	71
5	72
6	74
7	66
8	76
9	71
A	68
B	71
C	74
D	74
E	73
F	72
TOTAL	1150

16 PAKET DATA-6	
0	69
1	71
2	72
3	73
4	77
5	68
6	71
7	70
8	72
9	71
A	74
B	71
C	74
D	76
E	73
F	73
TOTAL	1155

16 PAKET DATA-7	
0	74
1	71
2	75
3	74
4	76
5	76
6	72
7	71
8	73
9	74
A	78
B	74
C	75
D	79
E	69
F	69
TOTAL	1180

16 PAKET DATA-8		16 PAKET DATA-9		16 PAKET DATA-A		16 PAKET DATA-B	
0	74	0	67	0	76	0	75
1	72	1	68	1	69	1	75
2	69	2	69	2	72	2	71
3	76	3	72	3	74	3	76
4	72	4	69	4	73	4	73
5	73	5	72	5	67	5	68
6	79	6	74	6	73	6	76
7	74	7	72	7	70	7	71
8	74	8	71	8	76	8	71
9	72	9	70	9	75	9	77
A	70	A	73	A	75	A	76
B	69	B	72	B	68	B	65
C	74	C	66	C	72	C	73
D	72	D	70	D	68	D	72
E	71	E	75	E	74	E	70
F	73	F	75	F	74	F	72
TOTAL	1164	TOTAL	1135	TOTAL	1156	TOTAL	1161
16 PAKET DATA-C		16 PAKET DATA-D		16 PAKET DATA-E		16 PAKET DATA-F	
0	72	0	72	0	72	0	70
1	70	1	74	1	70	1	71
2	64	2	74	2	71	2	74
3	75	3	70	3	71	3	73
4	76	4	74	4	74	4	72
5	73	5	73	5	69	5	71
6	70	6	74	6	70	6	70
7	73	7	77	7	70	7	71
8	70	8	77	8	72	8	67
9	78	9	75	9	71	9	70
A	71	A	72	A	76	A	70
B	70	B	70	B	73	B	73
C	71	C	71	C	77	C	77
D	69	D	76	D	73	D	73
E	71	E	69	E	73	E	67
F	72	F	74	F	67	F	75
TOTAL	1145	TOTAL	1172	TOTAL	1149	TOTAL	1144

Tabel 3.3.1.5. Detail Hasil Stratified Sampling.

Hasil *Stratified Sampling* pada setiap subfolder ditotalkan sehingga menghasilkan jumlah sampel total yang digunakan

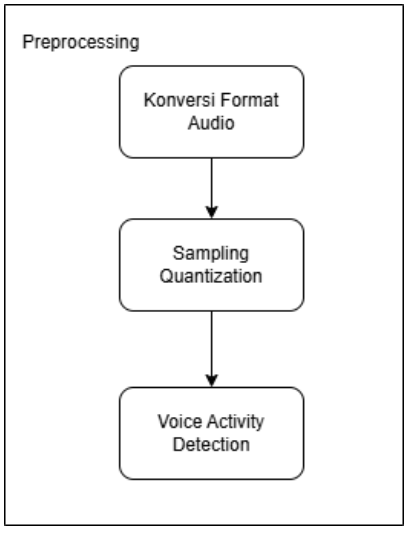
untuk pembuatan model. Total sampel yang digunakan adalah 18.501 data wicara.

3.3.2. *Data Splitting*.

Data sampel yang telah diambil menggunakan stratified Sampling disatukan dan diberlakukan data splitting yang membagi data menjadi tiga bagian yakni data pelatihan, data validasi, dan data uji. Perbandingan data antara pelatihan, validasi, dan data uji adalah 70% : 15% : 15% . Jumlah total data adalah 18.501, sehingga data latih akan berjumlah 12.951, data validasi berjumlah 2.775, dan data uji berjumlah 2.775.

3.3.3. *Preprocessing*.

Tahap persiapan data terdiri atas konversi berkas suara, *sampling* dan *quantization*, dan Voice Activity Detection dapat dilihat pada gambar 3.3.3.1.

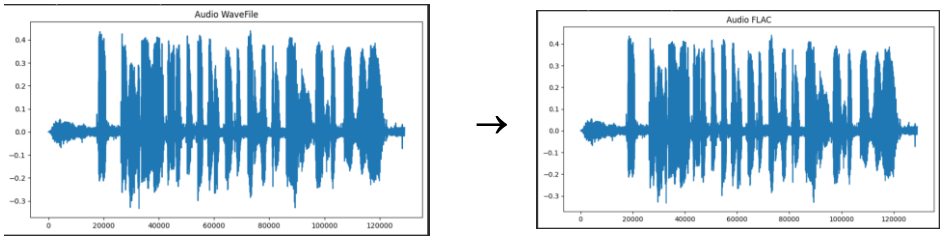


Gambar 3.3.3.1. Tahapan Preprocessing

3.3.3.1. Konversi Audio FLAC ke WAV.

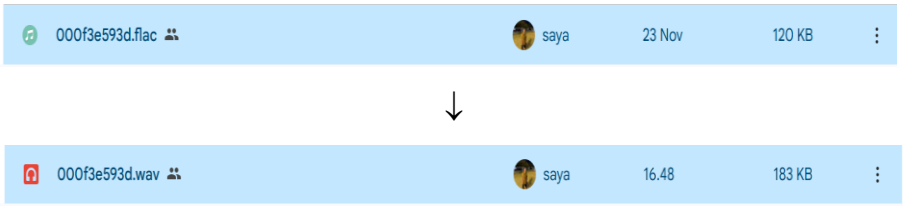
Format *lossless compressed* seperti FLAC merupakan format yang tidak menghilangkan informasi data pada saat kompresi dari format

rekaman suara mentah. Namun, pada proses ekstraksi fitur diperlukan format rekaman suara mentah seperti WAV sehingga perlu dilakukan konversi dari format FLAC ke WAV. Contoh dari konversi rekaman suara FLAC ke WAV dapat dilihat pada Gambar 3.3.3.2.



Gambar 3.3.3.2 Konversi Rekaman Suara FLAC ke WAV.

Hasil konversi audio tidak menunjukan perbedaan dalam grafik dikarenakan FLAC merupakan format lossless yakni format kompresi audio tanpa kehilangan data. Perbedaan yang dapat diamati yakni ukuran penyimpanan yang dibutuhkan oleh format FLAC lebih kecil dibandingkan format WAV. Perbedaan ukuran penyimpanan dari rekaman suara identik namun berbeda format data dapat dilihat pada gambar 3.3.3.3.



Gambar 3.3.3.3 Perbedaan Ukuran Penyimpanan

Rekaman format FLAC berukuran 120 KB yang lebih kecil dibandingkan rekaman format WAV berukuran 183 KB.

3.3.3.2. *Sampling* dan Quantization.

Sampling digunakan mendapatkan deretan nilai real yakni nilai amplitudo atau *samples* dari proses yang disebut sampling rate. Hasil dari Sampling adalah nilai real yang sudah diskrit terhadap waktu tetapi merupakan nilai amplitudo kontinu. Sehingga, diperlukan quantization untuk menempatkan nilai amplitudo ke level tertentu. Ketelitian sinyal wicara yang digunakan adalah 16 bit sehingga setiap nilai akan dipetakan ke satu dari 65.536 level terdekat. Hasil dari quantization adalah nilai amplitudo diskrit yang sudah distandarisasi ke dalam 65.536 level. Contoh Sampling dan Quantization dapat dilihat pada tabel 3.3.3.2.1

Sampling	Quantization
3,644620E-04	0,0003662109
4,356895E-04	0,0004272461
4,617208E-04	0,0004577637
4,618590E-04	0,0004577637
3,899573E-04	0,0003967285
4,148397E-04	0,0004272461
5,153982E-04	0,0005187988
5,346136E-04	0,0005493164
5,162396E-04	0,0005187988
4,093032E-04	0,0003967285
1,726370E-04	0,0001831055
1,794831E-05	0,0000305176
-3,869251E-05	-0,0000305176
-1,529391E-04	-0,0001525879
-1,824075E-04	-0,0001831055
-1,856561E-05	-0,0000305176

1,033091E-04	0,0000915527
1,269404E-04	0,0001220703
3,065602E-04	0,0003051758
6,393479E-04	0,0006408691

Tabel 3.3.3.2.1. Contoh Hasil *Sampling* dan *Quantization*.

Hasil dari *sampling* merupakan nilai amplitudo kontinu yang diperoleh dari proses pengambilan nilai amplitudo per detiknya. *Samples* merupakan nilai kontinu sehingga ditempatkan ke satu dari 65.563 level yang berbeda. Detail proses *Quantization* sebagai berikut :

1. Penentuan level *Quantization* yang digunakan yakni 16 bit atau 65.536 level yang berada pada rentang −32768 hingga +32767.
2. Pembuatan kode PCM. Nilai hasil sampling yang bersifat kontinu dikalikan dengan nilai maksimum level yakni 32767 kemudian dibulatkan. PCM menghasilan nilai representasi level dari tiap nilai amplitudo kontinu terhadap 65.536 level kuantisasi. Contoh kalkulasi kode dapat dilihat pada 3.3.3.2.2.

$$\begin{aligned} \text{Kode} &= 3{,}644620\text{E} - 04 \times 32767 \\ \text{Kode} &= 12 \end{aligned} \qquad \textbf{(3.3.3.2.2.)}$$

3. Rekonstruksi kembali kode PCM menjadi float. Kode PCM direkonstruksi kembali untuk mendapatkan nilai amplitudo diskrit yang sudah terkuantisasi. Rekonstruksi

didapatkan dari nilai sinyal asli dibagi 32767. Contoh rekonstruksi sinyal dapat dilihat pada 3.3.3.2.3.

$$x = \frac{3{,}644620E}{32767} \qquad \textbf{(3.3.3.2.2.)}$$

$$x = 0{,}0003662109$$

3.3.3.3. Voice Activity *Detection*.

Voice Activity Detection merupakan metode pemberian batas kepada deretan nilai amplitudo. VAD kemudian membagi deretan nilai amplitudo menjadi unit kecil atau *frame.* Pada tiap frame diterapkan perhitungan energi, daya sinyal, estimasi daya, dan perhitungan rasio *signal to noise,* konversi ke logaritmik , kemudian membandingkan hasil rasio *signal to noise* terhadap ambang batas yang ditetapkan. Hasil dari VAD merupakan frame yang dianggap berisi suara manusia dan frame yang dianggap bukan suara manusia. Detail kalkulasi sebagai berikut :

1. Hitung energi per frame.

   Format audio 16 bit menghasilkan 480 sampel per 300 *milisecond* frame. Setiap nilai amplitudo diskrit yang telah terkuantisasi dipangkatkan dan ditotalkan yang menghasilkan energi per frame. Perhitungan energi per frame dapat dilihat pada 3.3.3.3.1.

$$\sum x^2 = [0{,}0000001341 + \cdots . + 0{,}0000004107]$$

$$\sum x^2 = 0{,}0000026934 \qquad \textbf{(3.3.3.3.1)}$$

2. Perhitungan Daya.

   Daya merupakan hasil kalkulasi dari total energi per frame dibagi jumlah sampel pada frame tersebut. Contoh dari perhitungan daya dapat dilihat pada 3.3.3.3.2.

$$P = \frac{\sum x^2}{N} = \frac{0{,}0000026934}{20} \qquad \textbf{(3.3.3.3.2)}$$
$$P = 0{,}0000001347$$

3. Perhitungan Estimasi Daya.

   Estimasi daya merupakan hasil kuadrat jarak antar level kuantisasi dibagi *quantization error*. Contoh dari perhitungan estimasi daya dapat dilihat pada 3.3.3.3.3.

$$P = \frac{(\frac{1}{32767})^2}{12} = \frac{0{,}0000000009}{12} \qquad \textbf{(3.3.3.3.3)}$$
$$P = 0{,}0000000001$$

4. Perhitungan Rasio *Signal to noise*.

   Rasio *Signal to noise* merupakan hasil dari daya dibagi estimasi daya. Contoh dari perhitungan Rasio *Signal to noise* dapat dilihat pada 3.3.3.3.4.

$$SNR = \frac{0{,}0000001347}{0{,}0000000001} = 1735{,}2 \qquad \textbf{(3.3.3.3.4)}$$

5. Konversi Rasio *Signal to noise* menjadi satuan logaritmik.

   Rasio intensitas suara diukur menggunakan satuan logaritmik yang disebut desibel. Konversi menjadi satuan logaritmik bertujuan untuk memudahkan interprestasi sinyal ke ambang batas dan pemutusan kategori sinyal. Konversi menjadi satuan logaritmik merupakan hasil dari logaritma basis 10. Contoh dari Konversi Rasio

Signal to noise menjadi satuan logaritmik dapat dilihat pada 3.3.3.3.5.

$$SNR = 10\ log_{10}(1735{,}2) \quad \textbf{(3.3.3.3.5)}$$

$$SNR = 3{,}2393495390$$

6. Perbandingan terhadap ambang batas.
   Ambang batas yang digunakan yakni 6 hingga 10 yang menunjukan perbandingan sinyal wicara ke *noise*. Jika hasil SNR lebih besar dibandingkan ambang batas maksimal maka *frame* termasuk *frame* wicara, namun jika lebih kecil dibandingkan ambang batas maka *frame* memiliki banyak *noise* dibandingkan wicara dan tidak termasuk *frame* wicara.

3.3.4. *Feature Extraction.*

3.3.4.1. *Pre emphasis.*

Pre-Emphasis diperlukan sebagai tahap pengubahan sinyal asli menggunakan formula pre-emphasis untuk proses *framing and windowing* hingga transformasi frekuensi. Pre-emphasis merupakan tahap pembentukan barisan vektor dari selisih kuantisasi terhadap hasil perkalian nilai a dengan kuantisasi sebelumnya. Contoh *pre-emphasis* dapat dilihat pada 3.3.4.1.1.

$$X(n) = y(n) - a \times y\ (n-1)$$
$$X(n) = 0{,}000427246 - 0{,}97\ \times 0{,}000366211$$

(3.3.4.1.1)

Nilai y(n-1) merupakan nilai kuantisasi pertama pada sebuah frame dan a merupakan koefisien tetap

dengan nilai 0, 97. Hasil *pre-emphasis* untuk 20 nilai kuantisasi dapat dilihat pada tabel 3.3.4.1.2.

n	y(n) sebelum pre-emphasis	y(n) setelah pre-emphasis
0	0,0003662109	0,0003662109
1	0,0004272461	0,0000720215
2	0,0004577637	0,0003879028
3	0,0004577637	0,0000814979
4	0,0003967285	0,0003176755
5	0,0004272461	0,0001191008
6	0,0005187988	0,0004032710
7	0,0005493164	0,0001581435
8	0,0005187988	0,0003653996
9	0,0003967285	0,0000422909
10	0,0001831055	0,0001420833
11	0,0000305176	-0,0001073032
12	-0,0000305176	0,0000735666
13	-0,0001525879	-0,0002239475
14	-0,0001831055	0,0000341236
15	-0,0000305176	-0,0000636174
16	0,0000915527	0,0001532616
17	0,0001220703	-0,0000265935
18	0,0003051758	0,0003309715
19	0,0006408691	0,0003198268

Tabel 3.3.4.1.2 Nilai y(n) sebelum dan sesudah *Pre-Emphasis*

3.3.4.2. *Frame Blocking*

Sinyal wicara dipecah menjadi potongan frame berdurasi 20 hingga 30 milidetik yang membuat karakteristik sinyal wicara terlihat lebih stabil.

3.3.4.3. *Windowing.*

Setelah frame terbentuk, fungsi *windowing* dikalkulasikan dengan tiap frame dengan koefisien *window* untuk memperhalus tepi sinyal yang menghasilkan frame dengan bentuk sinyal yang lebih

teratur. Sehingga, menghasilkan frame yang lebih akurat pada tahap ekstraksi fitur selanjutnya. Penerapan fungsi hamming dapat dilihat pada 3.3.4.3.1.

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right)$$
$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi(0)}{20-1}\right) \quad \textbf{(3.3.4.3.1)}$$
$$w(n) = 0{,}0003662109$$

Setelah nilai hasil fungsi hamming didapatikan, langkah selanjutnya adalah menghitung nilai window melalui hasil kali w(n) dengan n. Perhitungan nilai window dapat dilihat pada 3.3.4.3.2.

$$y_w(n) = y(n) \times w(n)$$
$$y_w(n) = 0{,}0003662109 \times 0{,}0003662109 \quad \textbf{(3.3.4.3.2)}$$

Hasil windowing dapat dilihat pada tabel 3.3.4.3.3.

Penerapan Fungsi Hamming			Kalkulasi n x hasil hamming	
n	y(n)	w(n)	n	y(n) x w(n)
0	0,0003662109	0,291461	0	0,0001067362
1	0,0000720215	0,184012	1	0,0000132528
2	0,0003879028	0,11141	2	0,0000432161
3	0,0000814979	0,080761	3	0,0000065818
4	0,0003176755	0,095065	4	0,0000302000
5	0,0001191008	0,152923	5	0,0000182133
6	0,0004032710	0,248671	6	0,0001002818
7	0,0001581435	0,372936	7	0,0000589774
8	0,0003653996	0,513554	8	0,0001876526
9	0,0000422909	0,656761	9	0,0000277750
10	0,0001420833	0,788539	10	0,0001120382
11	-0,0001073032	0,895988	11	-0,0000961424
12	0,0000735666	0,96859	12	0,0000712559

13	-0,0002239475	0,999239		13	-0,0002237771
14	0,0000341236	0,984935		14	0,0000336095
15	-0,0000636174	0,927077		15	-0,0000589782
16	0,0001532616	0,831329		16	0,0001274108
17	-0,0000265935	0,707064		17	-0,0000188033
18	0,0003309715	0,566446		18	0,0001874773
19	0,0003198268	0,423239		19	0,0001353630

Tabel 3.3.4.3.3. Hasil *Hamming* dan *Windowing*.

3.3.4.4. *FFT.*

Setiap frame sinyal dihitung menggunakan algortima FFT yang menghasilkan distribusi energi berdasarkan frekuensi. Tahap pertama dari FFT adalah *zero padding* untuk memperpanjang sinyal pada domain waktu dengan aturan pangkat 2. Zero Padding dapat dilihat pada tabel 3.3.4.4.1.

Zero Padding	
n	y(n) x w(n)
0	0,0001067362
1	0,0000132528
2	0,0000432161
3	0,0000065818
4	0,0000302000
5	0,0000182133
6	0,0001002818
7	0,0000589774
8	0,0001876526
9	0,0000277750
10	0,0001120382
11	-0,0000961424
12	0,0000712559
13	-0,0002237771
14	0,0000336095
15	-0,0000589782
16	0,0001274108
17	-0,0000188033
18	0,0001874773

19	0,0001353630
20	0
21	0
22	0
23	0
24	0
25	0
26	0
27	0
28	0
29	0
30	0
31	0

Tabel 3.3.4.4.1. Zero Padding

Tahap kedua dalam FFT adalah konstruksi frekuensi bin untuk rFFT dengan membagi 2 nilai x *zero padding* dan ditambah 1. Hasil frekuensi bin untuk rFFT dapat dilihat pada tabel 3.3.4.4.2.

n	f[n]
0	0
1	250
2	500
3	750
4	1000
5	1250
6	1500
7	1750
8	2000
9	2250
10	2500
11	2750
12	3000
13	3250
14	3500
15	3750
16	4000

Tabel 3.3.4.4.2. Frekuensi bin rFFT.

Tahap ketiga adalah spektrum kompleks merupakan tahapan memperoleh nilai real dan imajiner dari tiap kumpulan nilai kontribusi real dan kontribusi imajiner melalui rumus cosinus dan sinus. Hasil nilai real dan imajiner pertama dari 32 iterasi dapat dilihat pada 3.3.4.4.3.

n	y(n) x w(n)	cos(2phin/32)	sin(2phin/32)	kontribusi real	kontribusi imaginer
0	0,0001067362	1,0000000000	0,0000000000	0,0001067362	0,0000000000
1	0,0000132528	0,9807852804	0,1950903220	0,0000129982	-0,0000025855
2	0,0000432161	0,9238795325	0,3826834324	0,0000399265	-0,0000165381
3	0,0000065818	0,8314696123	0,5555702330	0,0000054726	-0,0000036567
4	0,0000302000	0,7071067812	0,7071067812	0,0000213546	-0,0000213546
5	0,0000182133	0,5555702330	0,8314696123	0,0000101188	-0,0000151438
6	0,0001002818	0,3826834324	0,9238795325	0,0000383762	-0,0000926483
7	0,0000589774	0,1950903220	0,9807852804	0,0000115059	-0,0000578442
8	0,0001876526	0,0000000000	1,0000000000	0,0000000000	-0,0001876526
9	0,0000277750	-0,1950903220	0,9807852804	-0,0000054186	-0,0000272413
10	0,0001120382	-0,3826834324	0,9238795325	-0,0000428752	-0,0001035098
11	-0,0000961424	-0,5555702330	0,8314696123	0,0000534139	0,0000799395
12	0,0000712559	-0,7071067812	0,7071067812	-0,0000503855	-0,0000503855
13	-0,0002237771	-0,8314696123	0,5555702330	0,0001860638	0,0001243239
14	0,0000336095	-0,9238795325	0,3826834324	-0,0000310511	-0,0000128618
15	-0,0000589782	-0,9807852804	0,1950903220	0,0000578450	0,0000115061
16	0,0001274108	-1,0000000000	0,0000000000	-0,0001274108	0,0000000000
17	-0,0000188033	-0,9807852804	-0,1950903220	0,0000184420	-0,0000036683
18	0,0001874773	-0,9238795325	-0,3826834324	-0,0001732065	0,0000717445
19	0,0001353630	-0,8314696123	-0,5555702330	-0,0001125503	0,0000752037
20	0	-0,7071067812	-0,7071067812	0,0000000000	0,0000000000
21	0	-0,5555702330	-0,8314696123	0,0000000000	0,0000000000

22	0	-0,3826834324	-0,9238795325	0,0000000000	0,0000000000
23	0	-0,1950903220	-0,9807852804	0,0000000000	0,0000000000
24	0	0,0000000000	-1,0000000000	0,0000000000	0,0000000000
25	0	0,1950903220	-0,9807852804	0,0000000000	0,0000000000
26	0	0,3826834324	-0,9238795325	0,0000000000	0,0000000000
27	0	0,5555702330	-0,8314696123	0,0000000000	0,0000000000
28	0	0,7071067812	-0,7071067812	0,0000000000	0,0000000000
29	0	0,8314696123	-0,5555702330	0,0000000000	0,0000000000
30	0	0,9238795325	-0,3826834324	0,0000000000	0,0000000000
31	0	0,9807852804	-0,1950903220	0,0000000000	0,0000000000

Tabel 3.3.4.4.3 Kalkulasi Nilai Kontribusi.

Hasil kalkulasi nilai kontribusi real dan imajiner masing-masingnya merupakan total kontribusi dari real atau imajiner. Tabel 3.3.4.4.4 menampilkan daftar total kontribusi real dan imajiner untuk tiap iterasi.

i	Daftar Total Kontribusi	
	X[n] Real	X[n] Imaginer
1	0,0000193556	-0,0002323729
2	-0,0000580073	-0,0005659110
3	-0,0001211844	0,0007474690
4	0,0004490266	-0,0004022989
5	0,0000712218	0,0000341046
6	-0,0000445498	-0,0000665726
7	0,0001485880	-0,0001009379
8	0,0000466324	0,0002291409
9	0,0000896610	-0,0003120961
10	0,0000640238	0,0002776320
11	0,0000599039	-0,0001010309
12	0,0001916610	0,0000153819
13	0,0000234836	0,0001486836

14	0,0002245111	-0,0003859300
15	-0,0004564268	0,0005940395
16	0,0011374160	0,0000000000
17	-0,0004564268	-0,0005940395
18	0,0002245111	0,0003859300
19	0,0000234836	-0,0001486836
20	0,0001916610	-0,0000153819
21	0,0000599039	0,0001010309
22	0,0000640238	-0,0002776320
23	0,0000896610	0,0003120961
24	0,0000466324	-0,0002291409
25	0,0001485880	0,0001009379
26	-0,0000445498	0,0000665726
27	0,0000712218	-0,0000341046
28	0,0004490266	0,0004022989
29	-0,0001211844	-0,0007474690
30	-0,0000580073	0,0005659110
31	0,0000193556	0,0002323729
32	0,0008623409	0,0000000000

Tabel 3.3.4.4.4 Daftar Total Kontribusi.

Total nilai kontribusi atau spektrum kompleks yang diperoleh diubah menjadi daya untuk melakukan representasi berbasis energi. Proses konversi spektrum kompleks menjadi daya melalui perhitungan magnitude kuadrat, dan normalisasi NFFT. Perhitungan magnitude kuadrat dapat dilihat pada 3.3.4.4.5.

$$\begin{aligned} \mathrm{X[K]} &= a^2 + b^2 \\ \mathrm{X[K]} &= 0{,}0000193556^2 + -0{,}0002323729^2 \\ \mathrm{X[K]} &= 0{,}0000000544 \end{aligned} \qquad \textbf{(3.3.4.4.5)}$$

Tabel 3.3.4.4.6 merupakan tabel hasil perhitungan magnitude kuadrat untuk tiap spektrum kompleks.

i	X[n] Real	X[n] Imaginer	Hasil

1	0,0000193556	-0,0002323729	0,0000000544
2	-0,0000580073	-0,0005659110	0,0000003236
3	-0,0001211844	0,0007474690	0,0000005734
4	0,0004490266	-0,0004022989	0,0000003635
5	0,0000712218	0,0000341046	0,0000000062
6	-0,0000445498	-0,0000665726	0,0000000064
7	0,0001485880	-0,0001009379	0,0000000323
8	0,0000466324	0,0002291409	0,0000000547
9	0,0000896610	-0,0003120961	0,0000001054
10	0,0000640238	0,0002776320	0,0000000812
11	0,0000599039	-0,0001010309	0,0000000138
12	0,0001916610	0,0000153819	0,0000000370
13	0,0000234836	0,0001486836	0,0000000227
14	0,0002245111	-0,0003859300	0,0000001993
15	-0,0004564268	0,0005940395	0,0000005612
16	0,0011374160	0,0000000000	0,0000012937
17	-0,0004564268	-0,0005940395	0,0000005612
18	0,0002245111	0,0003859300	0,0000001993
19	0,0000234836	-0,0001486836	0,0000000227
20	0,0001916610	-0,0000153819	0,0000000370
21	0,0000599039	0,0001010309	0,0000000138
22	0,0000640238	-0,0002776320	0,0000000812
23	0,0000896610	0,0003120961	0,0000001054
24	0,0000466324	-0,0002291409	0,0000000547
25	0,0001485880	0,0001009379	0,0000000323
26	-0,0000445498	0,0000665726	0,0000000064
27	0,0000712218	-0,0000341046	0,0000000062
28	0,0004490266	0,0004022989	0,0000003635

29	-0,0001211844	-0,0007474690	0,0000005734
30	-0,0000580073	0,0005659110	0,0000003236
31	0,0000193556	0,0002323729	0,0000000544
32	0,0008623409	0,0000000000	0,0000007436

Tabel 3.3.4.4.6 Hasil Magnitude Kuadrat

Pada tabel hasil magnitude kuadrat hasil didapatkan dari hasil penjumlahan kuadrat X[n] real dan kuadrat X[n] imajiner. Tahapan selanjutnya dalah normalisasi NFFT dengan membagi nilai magnitude kuadrat dengan panjang NFFT yakni 32. Perhitungan normalisasi NFFT dapat dilihat pada 3.3.4.4.7.

$$P[\mathrm{K}] = \frac{\mathrm{X[K]}}{32}$$

$$P[\mathrm{K}] = \frac{0{,}0000000544}{32} \qquad \textbf{(3.3.4.4.7)}$$

$$P[\mathrm{K}] = 0{,}0000000017$$

Pada Tabel 3.3.4.4.8 menampilkan daftar nilai normalisasi NFFT. Hasil dari nilai NFFT merupakan nilai daya.

NORMALISASI NFFT		
	X[k]	P[k]
i	0,0000000544	0,0000000017
1	0,0000003236	0,0000000101
2	0,0000005734	0,0000000179
3	0,0000003635	0,0000000114
4	0,0000000062	0,0000000002
5	0,0000000064	0,0000000002
6	0,0000000323	0,0000000010
7	0,0000000547	0,0000000017
8	0,0000001054	0,0000000033
9	0,0000000812	0,0000000025
10	0,0000000138	0,0000000004

11	0,0000000370	0,0000000012
12	0,0000000227	0,0000000007
13	0,0000001993	0,0000000062
14	0,0000005612	0,0000000175
15	0,0000012937	0,0000000404
16	0,0000005612	0,0000000175
17	0,0000001993	0,0000000062
18	0,0000000227	0,0000000007
19	0,0000000370	0,0000000012
20	0,0000000138	0,0000000004
21	0,0000000812	0,0000000025
22	0,0000001054	0,0000000033
23	0,0000000547	0,0000000017
24	0,0000000323	0,0000000010
25	0,0000000064	0,0000000002
26	0,0000000062	0,0000000002
27	0,0000003635	0,0000000114
28	0,0000005734	0,0000000179
29	0,0000003236	0,0000000101
30	0,0000000544	0,0000000017
31	0,0000007436	0,0000000232
32	1,0000007436	0,0312500232

Tabel 3.3.4.4.8. Daftar Normalisasi NFFT.

3.3.4.5. *Mel filter bank.*

Memproyeksikan energi ke skala mel yang lebih sesuai dengan persepsi manusia menggunakan filter mel melalui pemetaaan yang mengubah distribusi frekuensi hasil FFT menjadi distribusi logaritmik. Pengubahan dilakukan dengan mengkonversi nilai frekuensi ke skala mel menggunakan fungsi mel. Hasil dari penerapan fungsi mel adalah batas-batas filter yang menentukan rentang setiap filter, agar setiap filter dapat memberikan bobot tertentu pada frekuensi hasil FFT. Hasil dari penerapan filter adalah representasi energi yang sudah diproyeksikan ke domain mel yang mencerminkan sensitivitas pendengaran manusia.

Proses dari penerapan *Mel Filter Bank,* sebagai berikut :

1. Tahap pertama yakni konversi nilai frekuensi bin FFT menjadi skala mel menggunakan formula yang dapat dilihat pada 3.3.4.5.1.

$$\boldsymbol{Mel(f) = 1127\, In\left(1 + \frac{f}{700}\right)}$$

$$\boldsymbol{Mel(f) = 1127\, In\left(1 + \frac{250}{700}\right)} \qquad \textbf{(3.3.4.5.1)}$$

$$\boldsymbol{Mel(f) = 1127\, ln\,(1,357142857)}$$

$$\boldsymbol{Mel(f) = 344,165119}$$

Pada tabel 3.3.4.5.2. menampilan daftar hasil konversi nilai frekuensi bin FFT menjadi skala mel.

n	f[n]	mel
0	0	0
1	250	344,1633419
2	500	607,4459197
3	750	820,720552
4	1000	999,9855371
5	1250	1154,610403
6	1500	1290,557443
7	1750	1411,856575
8	2000	1521,359554
9	2250	1621,158718
10	2500	1712,83478
11	2750	1797,611138
12	3000	1876,45406
13	3250	1950,140049
14	3500	2019,302495
15	3750	2084,464815
16	4000	2146,064528

Tabel 3.3.4.5.2. Hasil Konversi Mel

2. Tahap kedua adalah penentuan jumlah *mel filter* yakni 12 filter mel, sehingga diperlukan 14 titik batas.
3. Tahap ketiga adalah perhitungan jarak *mel filter* dengan mencari selisih nilai maksimum dan minimum dibagi hasil dari jumlah *mel filter* ditambah 1. Perhitungan jarak *mel filter* dapat dilihat pada 3.3.4.5.3.

$$\Delta mel = \frac{max - min}{n - filter + 1}$$
$$\Delta mel = \frac{2146,064528}{13} \quad \textbf{(3.3.4.5.3)}$$
$$\Delta mel = 165,0818867$$

4. Tahap keempat adalah pembuatan titik mel dari hasil kali antara jarak filter mel ke tiap indeks titik batas. Perhitungan titik mel dapat dilihat pada 3.3.4.5.4.

$$mel_n = \Delta mel \times n$$
$$mel_n = 165,0818867 \times 1 \quad \textbf{(3.3.4.5.4)}$$
$$mel_n = 165,0818867$$

Pada tabel 3.3.4.5.5 menampilkan daftar lengkap hasil perhitungan titik mel.

n	mel_n
0	0
1	165,0819
2	330,1638
3	495,2457
4	660,3275
5	825,4094
6	990,4913
7	1155,573
8	1320,655
9	1485,737
10	1650,819

11	1815,901
12	1980,983
13	2146,065

Tabel 3.3.4.5.5. Hasil Perhitungan Titik Mel

5. Tahapan kelima adalah konversi mel ke satuan FFT yakni Hz agar dapat dikalkulasikan dengan bin frekuensi dan energi spektrum. Konversi mel ke Hz dapat dilihat pada 3.3.4.5.6.

$$f_n = 700(10^{2595mel} - 1)$$
$$f_n = 700(10^{2595 \times 330{,}1637735} - 1) \qquad \textbf{(3.3.4.5.6)}$$
$$f_n = 238{,}2720539$$

Pada tabel 3.3.4.5.7. menampilkan daftar hasil konversi mel ke hz.

mel	f_n
0	0
165,0818867	110,4260841
330,1637735	238,2720539
495,2456602	386,2859235
660,3275469	557,6492103
825,4094337	756,0453209
990,4913204	985,7387253
1155,573207	1251,66662
1320,655094	1559,545052
1485,736981	1915,991783
1650,818867	2328,668538
1815,900754	2806,44569
1980,982641	3359,592928
2146,064528	4000

Tabel 3.3.4.5.7 Hasil Konversi Mel ke Hz.

6. Tahapan keenam adalah pembuatan filter segitiga yang menerapkan rumus filter segitiga atau fungsi bobot. Penerapan filter segitiga dilakukan membandingkan nilai bin FFT terhadap hasil konversi pada tahap

kelima untuk mendapatkan rentang pada fungsi bobot. Setelah mendapatkan rentang yang sesuai, maka kalkulasi dengan formula rentang yang sesuai. Tabel 3.3.4.5.8. menampilkan daftar hasil perbandingan nilai bin FFT terhadap hasil konversi pada tahap kelima.

n	x	x+1	x+2	Nilai			
1	0	110,4261	238,2721	0			
2	110,4261	238,2721	386,2859	250			
3	238,2721	386,2859	557,6492	250	500		
4	386,2859	557,6492	756,0453	500	750		
5	557,6492	756,0453	985,7387	750			
6	756,0453	985,7387	1251,667	1000	1250		
7	985,7387	1251,667	1559,545	1000	1250	1500	
8	1251,667	1559,545	1915,992	1500	1750		
9	1559,545	1915,992	2328,669	1750	2000	2250	
10	1915,992	2328,669	2806,446	2000	2250	2500	2750
11	2328,669	2806,446	3359,593	2500	2750	3000	3250
12	2806,446	3359,593	4000	3000	3250		

Tabel 3.3.4.5.8. Perbandingan nilai bin FFT dan Hasil Konversi Mel

Pada tabel 3.3.4.5.8. x merupakan nilai rentang kiri, x+1 merupakan nilai rentang tengah, dan x+2 merupakan nilai rentang kanan. Ketiga nilai x merupakan nilai hasil konversi mel ke Hz sedangkan kolom nilai merupakan nilai bin FFT yang termasuk pada ketiga rentang nilai x. Setelah mendapatkan nilai yang termasuk pada tiap rentang, maka penerapan kalkulasi filter segitiga dapat dilakukan. Tiap nilai bin yang termasuk pada rentang diidentifikasi pada rentang yang sesuai, bila termasuk pada rentang bawah (x hingga x+1) maka gunakan formula sisi naik dan jika sebaliknya gunakan formula sisi

turun. Tabel 3.3.4.5.9. menampilkan formula sisi naik dan turun.

Sisi Naik	Sisi Turun
$\frac{f - f_{m-1}}{f_m - f_{m-1}}$	$\frac{f_{m+1} + f_{m-1}}{f_{m+1} - f_m}$

Tabel 3.3.4.5.9 Formula Fungsi Bobot

Pada tabel 3.3.4.5.10 menampilkan pembagian rentang nilai bin FFT.

n	x	x+1	x+2	BAWAH		ATAS		
1	0	110,4261	238,2721					
2	110,4261	238,2721	386,2859			250		
3	238,2721	386,2859	557,6492	250		500		
4	386,2859	557,6492	756,0453	500		750		
5	557,6492	756,0453	985,7387	750				
6	756,0453	985,7387	1251,667			1000	1250	
7	985,7387	1251,667	1559,545	1000	1250	1500		
8	1251,667	1559,545	1915,992	1500		1750		
9	1559,545	1915,992	2328,669	1750		2000	2250	
10	1915,992	2328,669	2806,446	2000		2250	2500	2750
11	2328,669	2806,446	3359,593	2500	2750	3000	3250	
12	2806,446	3359,593	4000	3000	3250			

Tabel 3.3.4.5.10 Pemetaan bin FFT pada Rentang Bobot.

Pada tabel 3.3.4.5.11 menampikan hasil dari penerapan bin FFT pada rentang bobot menggunakan formula fungsi bobot.

Fungsi Bobot pada Bin FFT				
		147,366680 6		
11,11111785		170,466665 7		
113,0213723		197,404106 6		
191,6132027				

		265,128959 8	264,929226 1	
13,47373376	263,261274 7	306,916612 9		
247,5307957		355,533366 2		
189,640986		411,817895 2	411,710537 7	
83,18543281		476,975426 5	476,886345 9	476,797265 2
170,5017047	420,331461 9	552,254272 7	552,179859	
192,7189567	442,554309 6			

Tabel 3.3.4.5.11 Fungsi Bobot pada Bin FFT.

7. Tahap ketujuh adalah pembuatan energi mel dari hasil kali antara energi spektrum dan hasil fungsi bobot pada bin FFT, kemudian mentotalkan hasil dengan bobot bin yang serupa. Tabel 3.3.4.5.12 menampilkan daftar energi mel.

REKAP HASIL ENERGI MEL					TOTAL
					0,000000000
		0,000001 4903			0,000001490 3
0,000000 1124		0,000003 0545			0,000003166 9
0,000002 0252		0,000002 2422			0,000004267 4

0,00 0002 1764					0,000 00217 64
		0,00 0000 0517	0,00 0000 0531		0,000 00010 48
0,00 0000 0026	0,00 0000 0528	0,00 0000 3095			0,000 00036 49
0,00 0000 2496		0,00 0000 6075			0,000 00085 71
0,00 0000 3240		0,00 0001 3570	0,00 0001 0444		0,000 00272 55
0,00 0000 2741		0,00 0001 2100	0,00 0000 2056	0,00 0000 5509	0,000 00224 06
0,00 0000 0735	0,00 0000 4856	0,00 0000 3910	0,00 0003 4399		0,000 00439 00
0,00 0000 1365	0,00 0002 7569				0,000 00289 34

Tabel 3.3.4.5.12. Daftar Energi Mel.

3.3.4.6. *Logarithm*

Penerapan fungsi logaritma pada hasil dari representasi Mel yang mengubah nilai energi representasi mel menjadi rentang yang lebih kecil. Proses logarithmic compression mengecilkan rentang energi yang besar dan membuat perubahan energi kecil menjadi lebih menonjol sehingga pola-pola akustik penting dapat dikenali. Tabel 3.3.4.6.1 menampilkan hasil penerapan fungsi logaritma menggunaan logaritma basis 10.

LOGARITHM		
n-Filter	Total	Log10
1	0	-12
2	1,49034E-06	-5,82672
3	3,16689E-06	-5,49937
4	4,26738E-06	-5,36984
5	2,17642E-06	-5,66226

6	1,04788E-07	-6,97969
7	3,6489E-07	-6,43784
8	8,57114E-07	-6,06696
9	2,72547E-06	-5,56456
10	2,24056E-06	-5,64964
11	4,39002E-06	-5,35753
12	2,89339E-06	-5,53859

Tabel 3.3.4.6.1. Fungsi Logarima.

Pada tabel 3.3.4.6.1 n-filter merupakan jumlah filter yang telah ditentukan sebelumnya yakni berjumlah 12 filter, total merupakan total energi mel pada tiap bin FFT yang serupa, dan log10 merupakan hasil kalkulasi fungsi logaritma pada total energi mel.

3.3.4.7. *DCT.*

Deret logaritma mel diproyeksikan ke basis fungsi kosinus menggunakan formula DCT dan menghasilkan representasi yang terpisah antar komponen yang bertujuan untuk menghilangkan korelasi mel yang tumpang tindih. Hasil dari DCT adalah nilai yang menggambarkan garis besar pola energi suara pada berbagai frekuensi atau yang disebut koefisien cepstral. Tahapan penerapan DCT sebagai berikut :

1. Tahap pertama adalah membuat nilai C untuk tiap filter. Nilai C merupakan hasil kalkulasi nilai pi dikali indeks filter dibagi jumlah filter dikali selisih indeks filter dengan 0,5.
2. Tahap kedua adalah menghitung nilai kosinus dari tiap nilai C.
3. Tahap ketiga adalah menghitung nilai kontribusi yakni hasil perkalian antara nilai C dan kosinus dari C.

Tabel 3.3.4.7.1. menampilkan penerapan DCT pada indeks filter 1.

K	C1	Cos	kontribusi (log x cos)
1	0,1309	0,991445	-11,89733834
2	0,392699	0,92388	-5,383182925
3	0,654498	0,793353	-4,362940731
4	0,916298	0,608761	-3,268950404
5	1,178097	0,382683	-2,166851855
6	1,439897	0,130526	-0,911032411
7	1,701696	-0,13053	0,840306422
8	1,963495	-0,38268	2,32172561
9	2,225295	-0,60876	3,387488731
10	2,487094	-0,79335	4,482163359
11	2,748894	-0,92388	4,949715345
12	3,010693	-0,99144	5,491208855
		Total	-6,51768834

Tabel 3.3.4.7.1 Penerapan DCT pada indeks filter 1

Pada tabel 3.3.4.7.1 terdapat baris total yang merupakan total dari nilai kontribusi untuk C1 dan untuk perlu dilakukan kalkulasi dari tahap 1 hingga 3 untuk tiap indeks filter dan memperoleh nilai total kontribusi untuk tiap indeks. Selanjutnya nilai total kontribusi pada tiap filter diurutkan berdasarkan indeks filter yang membentuk nilai MFCC pada sebuah frame. Tabel 3.3.4.7.1 merupakan tabel MFCC.

MFCC	
n	Kontribusi Total
1	-6,5176883405
2	-3,6508365739
3	-6,5530574417
4	-7,6839953299
5	-5,0769993608
6	-3,1476301670
7	-3,0515416350
8	-3,6062968089
9	-2,6303652307

10	-1,6592616455
11	0,1842813267
12	0,0000000000

Tabel 3.3.4.7.1 Tabel MFCC.

3.3.4.8. *MFCC features, deltas* dan *double deltas..*

MFCC merepresentasikan amplop spektral sinyal wicara pada satu titik waktu sehingga bersifat statis. Oleh karena itu, diperlukan fitur delta untuk menggambarkan perubahan nilai MFCC dari satu frame ke frame berikutnya, dan fitur double delta untuk merepresentasikan percepatan perubahan dari satu frame ke frame berikutnya. Maka, MFCC tidak hanya menggambarkan karakter spektral pada satu waktu, tetapi juga pola perubahannya sepanjang sinyal wicara.

3.3.5. *Modelling.*

3.3.5.1. GMM.

Pada Automatic Speech Recognition (ASR), sebuah kata tersusun dari sejumlah fonem yang berfungsi sebagai unit bunyi terkecil. Sebuah fonem dimodelkan menjadi 3 *state* HMM (awal-tengah-akhir). Tiap *state* terdiri atas sebuah *Gaussian Mixture Model* (GMM) . GMM sebagai model probabilitas obervasi pada sebuah *state*, sedangkan HMM adalah penentu urutan *state* berdasarkan hasil probabilitas observasi. Tahapan dari algoritma GMM sebagai berikut :

1 Tahap pertama adalah pembuatan rangkaian vektor observasi kontinu. Rangkaian vektor observasi kontinu merupakan gabungan seluruh vektor fitur pada sebuah gelombang wicara. Pembuatan vektor observasi kotinu dapat dilihat pada 3.3.5.1.1.

$$x = \begin{bmatrix} 2{,}86 \\ 4{,}26 \\ 3{,}69 \\ 1{,}67 \\ 1{,}42 \\ 5{,}21 \\ 2{,}18 \\ 3{,}9 \\ 3{,}17 \\ 5{,}17 \\ 4{,}66 \\ 4{,}14 \\ 4{,}63 \\ 5{,}45 \\ 4{,}04 \\ 4{,}51 \\ 1{,}94 \\ 3{,}11 \\ 2{,}24 \\ 4{,}54 \end{bmatrix}$$ **(3.3.5.1.1**

2 Tahap kedua adalah perhitungan kecocokan komponen gaussian. Pada sebuah state terdapat beberapa komponen gaussian yang pada setiap komponen dilakukan perhitungan kedekatan vektor fitur dengan *mean* komponen. Hasil kedekatan dikalikan dengan bobot komponen dan dijumlahkan, sehingga menghasilkan total kontribusi komponen untuk sebuah vektor fitur. Perhitungan kecocokan komponen gaussian dapat dilihat pada 3.3.5.1.2.

$$p(x|b,\sigma^2) = w \times \frac{1}{\sqrt{2\pi\sigma^2}} \times \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad \textbf{(3.3.5.1.2.)}$$

$$p(x|b,\sigma^2) = 0{,}5 \times \frac{1}{\sqrt{2\pi 0{,}8}} \times \exp\left(-\frac{(2{,}86-2{,}8)^2}{2\times 0{,}8}\right)$$

$$p(x|b,\sigma^2) = 0{,}22269$$

Tabel 3.3.5.1.3 menampilkan daftar hasil kalkulasi kecocokan komponen gaussian menggunakan 2 kelompok komponen.

N	INP	Kecocokan Komponen-Set-1	Kecocokan Komponen-Set-2
1	2,86	0,22269	0,00257
2	4,26	0,09507	0,04949
3	3,69	0,16246	0,01794
4	1,67	0,13382	0,00006
5	1,42	0,10411	0,00002
6	5,21	0,02185	0,15069
7	2,18	0,19123	0,00035
8	3,9	0,13745	0,02688
9	3,17	0,21113	0,00567
10	5,17	0,02358	0,14589
11	4,66	0,05589	0,08637
12	4,14	0,10874	0,04085
13	4,63	0,05842	0,08320
14	5,45	0,01344	0,17808
15	4,04	0,12057	0,03450
16	4,51	0,06924	0,07115
17	1,94	0,16590	0,00016
18	3,11	0,21461	0,00489
19	2,24	0,19672	0,00042
20	4,54	0,06643	0,07407

Tabel 3.3.5.1.3 Daftar Kecocokan Komponen Gaussian.

3 Tahap ketiga adalah normalisasi menjadi derajat keanggotaan. Setiap kontribusi komponen dibagi dengan total kontribusi komponen sehingga nilai kontribusi komponen berada pada jangkauan 0 hingga 1. Normalisasi derajat keanggotaan memerlukan total probabilitas komponen yang dapat dilihat pada 3.3.5.1.4.

$$p(x_i) = \sum_{k=1}^{k} \pi_K \, N(x|\mu_K, \sigma_k^2) \quad \textbf{(3.3.5.1.4.)}$$

$$p(x_i) = 0{,}22269 + 0{,}00257$$

$$p(x_i) = 0{,}22527$$

Normalisasi menjadi derajat keanggotaan dapat dilihat pada 3.3.5.1.5.

$$\gamma_{ik} = \frac{p(x|b,\sigma^2)}{p(x_i)} \quad \textbf{(3.3.5.1.5.)}$$

$$\gamma_{ik} = \frac{0{,}22269}{0{,}22527}$$

$$\gamma_{ik} = 0{,}98858$$

Tabel 3.3.5.1.6 menampilkan daftar total probabilits dan normalisasi derajat keanggotaan.

Probabilitas Total Komponen	Normalisasi Derajat Keanggotaan	
0,22527	0,98858	0,01142
0,14456	0,65764	0,34236
0,18040	0,90054	0,09946
0,13388	0,99955	0,00045
0,10414	0,99977	0,00023
0,17253	0,12662	0,87338
0,19158	0,99819	0,00181
0,16432	0,83644	0,16356
0,21680	0,97385	0,02615
0,16948	0,13915	0,86085
0,14226	0,39289	0,60711
0,14959	0,72695	0,27305
0,14162	0,41251	0,58749
0,19152	0,07018	0,92982
0,15507	0,77751	0,22249
0,14039	0,49320	0,50680
0,16606	0,99905	0,00095
0,21950	0,97770	0,02230

0,19714	0,99787	0,00213
0,14050	0,47283	0,52717

Tabel 3.3.5.1.6 Daftar Probabilitas Total dan Normalisasi Komponen.

4 Tahap keempat adalah pembaharuan parameter. Pembaharuan parameter dilakukan dengan membaharui bobot komponen, membaharui *mean*, dan membaharui *covariance.* Pembaharuan parameter sebagai berikut :

1. Pembaharuan Bobot.

$$w_k^{new} = \frac{\sum_{t=1}^{T} \gamma_k(x_t)}{T} \quad \textbf{(3.3.5.1.7.)}$$

$$w_k^{new} = \frac{13{,}94100}{20}$$

$$w_k^{new} = 0{,}697049978$$

2. Pembaharuan *mean*.

$$\mu_k^{new} = \frac{\sum_{t=1}^{T} \gamma_k(x_t) x_t}{\sum_{t=1}^{T} \gamma_k(x_t)} \quad \textbf{(3.3.5.1.8.)}$$

$$\mu_k^{new} = \frac{43{,}8040746}{13{,}94100}$$

$$\mu_k^{new} = 3{,}142104295$$

3. Pembaharuan *covariance.*

$$(\sigma_k^2)^{new} = \frac{\sum_{t=1}^{T} \gamma_k(x_t)(x_t - \mu_k^{new})^2}{\sum_{t=1}^{T} \gamma_k(x_t)} \quad \textbf{(3.3.5.1.9.)}$$

$$(\sigma_k^2)^{new} = \frac{16{,}46727192}{13{,}94100}$$

$$(\sigma_k^2)^{new} = 1{,}181211709$$

Pada Tabel 3.3.5.1.10 menampilkan data pembaharuan parameter.

Pembaharuan Bobot		Pembaharuan *Mean*		Pembaharuan *Covariance*	
W1	W2	*Mean* 1	*Mean* 2	*Covariance* 1	*Covariance* 2
0,697049978	0,302950022	3,142104	4,783945	1,181211709	0,261228171

Tabel 3.3.5.1.10 Pembaharuan Parameter.

5 Tahap keenam adalah perhitungan nilai keanggotaan hingga stabil. Pengulangan perhitungan derajat keanggotaan setiap komponen sambil membaharui parameter komponen hingga perubahan nilai *likelihood* menjadi sangat kecil. Perubahan nilai likelihood didapat dari selisih total nilai probabilitas komponen sebelum dan sesudah pembaharuan parameter.

3.3.5.2. TDNN

Time Delay Neural Network terdiri atas *input layer*, *hidden layer,* dan output layer. Tahapan dari algoritma TDNN sebagai berikut :

1. Pada *input layer*, pola konteks diterapkan untuk memproses hasil ekstraksi fitur MFCC. Pola konteks yang digunakan adalah -2 hingga +2 yang menandakan pola yang diambil adalah 2 frame dari kiri hingga 2 frame ke kanan yang diukur dari frame utama. Selanjunya diberlakukan splicing untuk menghasilkan sebuah vektor gabungan dari setiap nilai komponen frame. Proses *splicing* membentuk sebuah vektor berukuran jumlah frame x jumlah komponen. Setelah vektor *splicing* terbentuk maka dapat melanjutkan pada tahapan kalkulasi antara vektor *weight* , vektor *splicing*, dan nilai *bias* untuk setiap neuron. Kemudian, fungsi ReLU diterapkan untuk mendapatkan nilai keluaran dari lapisan. Jika nilai dari komponen lebih besar dari 0 maka outputnya adalah nilai komponen dan jika tidak maka outputnya adalah 0. Hasil dari layer pertama adalah ringkasan atau vektor hasil dari penerapan fungsi ReLu per frame sesuai pola konteks.

2. *Hidden layer* memiliki pola konteks yang berbeda dan memerlukan hasil pada *input layer* sebagai nilai *input*. *Hidden layer* tidak menerapkan seluruh pola konteks seperti *input layer*. Melainkan menggunakan sub-konteks pola, yakni {-1 ,+2} untuk layer kedua, {-3, 3} untuk layer ketiga, {-7, 2} untuk layer keempat, dan {0} untuk layer kelima. Pola konteks pada masing masing layer memiliki fungsi yang berbeda. Layer kedua untuk membuat model mempelajari pola transisi unit fonem, layer ketiga untuk mengenali pola fonem utuh, dan layer keempat untuk mengenali kata secara utuh. Layer kelima untuk menyimpulkan hasil dari layer keempat. Proses kalkulasi pada *hidden layer* sama seperti *input layer* yang terdiri atas *splicing*, kalkulasi linear (*weight,* vektor *splicing,* dan *bias)* , dan penerapan fungsi ReLu.
3. *Ouput Layer* merupakan *layer* yang mengubah kalkulasi dari *layer* terakhir pada *hidden layer* menjadi sebuah vektor probabilitas state. Hasil pada *Output Layer* adalah persentase seberapa besar sebuah frame masuk ke sebuah state. Persentase probabilitas sebuah frame ke sebuah state dihitung menggunakan fungsi aktivasi *softmax.*

3.3.6. Lexicon-HMM.

3.3.6.1 Pembuatan Lexicon.

Melakukan persiapan data dengan mengubah transkrip data suara menjadi lexicon. Transkrip data akan melalui proses normalisasi, tokenisasi, pembentukan kamus, dan pembuatan lexicon. Pembuatan Lexicon terdiri atas beberapa tahapan yakni pembuatan daftar fonem, pembuatan kamus

yang berisi kata unik dari seluruh transkrip data, pemetaan huruf ke fonem, pembuatan lexicon awal, dan pembuatan lexicon lengkap.

3.3.6.2 Penerapan Lexicon-HMM

Model akustik GMM dan TDNN menghasilkan probabilitas sebuah frame menuju sebuah *state* yang disebut sebagai *likelihood* yang berfungsi sebagai *emission probability*. Nilai *likelihood* dibentuk menjadi rangkaian *likelihood* yang merepresentasikan urutan observasi. Rangkaian *likelihood* digunakan sebagai nilai untuk proses pembentukan urutan kata pada Lexicon-HMM. Setiap kata memiliki urutan fonem yang setiap fonem memiliki 3 *state* (awal-tengah-akhir), sehingga urutan fonem membentuk rantai *state* untuk sebuah kata.

Tahap selanjutnya yakni pembentukan tabel hubungan waktu-sate menggunakan nilai likelihood pada sebuah state pada waktu. Pada proses pembentukan tabel hubungan waktu-state menggunakan algoritma Viterbi dan mendapatkan skor untuk *state* pada waktu tertentu dan pointer ke *state* sebelumnya. Pada saat state terakhir didapatkan, maka backracking dilakukan dan menghasilkan urutan state terbaik. Urutan state terbaik dikumpulkan menjadi sebuah fonem dan dibandingkan dengan lexicon. Urutan fonem yang telah dibandingkan dengan lexicon dipetakan menjadi urutan kata.

3.3.7. *Language model.*

Language model yang digunakan pada penelitian mengenai kinerja model akustik klasik GMM dan model akustik modern TDNN dalam pengenalan bahasa Jawa adalah model N-Gram. Model N-Gram bekerja dengan beberapa tahapan berikut :

1. Penyusunan korpus.

   Hasil transkrip dari lexicon-HMM dibentuk menjadi korpus. Tiap baris transkrip diberlakukan sebagai sebuah kalimat mentah yang akan diproses.
2. Pembagian data.

   Pembagian data mengikuti pembagian data awal yang sudah ditetapkan.
3. Normalisasi.

   Data latih dan data uji di proses secara terpisah dengan melakukan standarisasi karakter menjadi karakter non kapital untuk memudahkan model pemrosesan kata dengan mengurangi ukuran kosakata dan memperbaiki estimasi probabilitas.
4. Pemecahan kata.

   Data yang telah dinormalisasi dipecah menjadi rangkaian satuan kata yang digunakan untuk pembentukan N-gram. Karena model N-gram bekerja melalui probabilitas urutan kata.
5. Pembentukan rangkaian N-gram dari satuan kata.

   Pengubahan pecahan kata menjadi hubungan antar kata. Proses ini merupakan pembentukan unigram, bigram, trigram, dan fourgram yang berfungsi untuk membantu mengetahui berapa jumlah sebuah kata muncul setelah kata lainnya.
6. Menghitung frekuensi N-gram.

Menghitung jumlah kombinasi kata berurutan yang merepresentasikan kemunculan kombinasi rangkaian kata tersebut pada korpus.

7. Estimasi probabilitas N-Gram menggunakan MLE.
   Estimasi probabilitas menggunakan MLE diperoleh dengan membandingkan probabilitas seberapa sering kombinasi kata dalam konteks dengan seluruh kombinasi dengan konteks yang sama.
8. Penerapan Smoothing dengan Kneser-Ney.
   Estimasi probabilitas mampu menghasilkan nilai yang timpang dikarenakan bergantung pada jumlah kombinasi pada konteks, sehingga rangkaian kata yang jarang muncul dapat memiliki nilai probabilitas 0. Smoothing bertujuan untuk mengubah probabilitas dengan nilai 0 menjadi nilai lebih dari 0 melalui pemindahan masa probabilitas dari selisih konstanta dan nilai dari probabilitas dengan nilai tinggi.
9. Penerapan Model setelah Smoothing.
   Pengaplikasian model setelah proses Smoothing pada data uji.

3.3.8. *Hyperparameter Tuning.*

Pada penelitian mengenai kinerja model akustik klasik GMM dan model akustik modern TDNN dalam pengenalan bahasa Jawa terdapat parameter yang dapat diatur untuk menghasilkan Word Error Rate terendah untuk masing-masing model. Parameter dan penerapannya akan dituliskan pada tabel 3.4.1 Tabel Desain Penelitian.

3.3.9. Evaluasi Model menggunakan *Word Error Rate.*

Tahap Evaluasi model menggunakan *Word Error Rate* yang menghitung tingkat kesalahan kombinasi kata berdasarkan

jumlah kesalahan substitusi, penghapusan, dan penyisipan kata. Hasil dari penjumlahan ketiga variabel dibagi dengan jumlah kata dari dokumen pembanding. Secara singkat, persentasi dari WER menggambarkan tingkat kesalahan dari hasil transkrip dan dokumen pembanding. Formula evaluasi *Word Error Rate* dapat dilihat pada 3.3.9.1.

$$\boldsymbol{WER = \frac{S + D + I}{N}} \qquad \textbf{(3.3.9.1)}$$

Detail dari *Word Error Rate* sebagai berikut :

1. Hasil transkripsi dari model di sandingan dengan transkripsi bawaan yang telah melalui proses pembersihan data untuk melihat perbedaan kata dari kedua transkripsi. Contoh perbandingan transkrip dapat dilihat pada tabel 3.3.9.2.

Transkrip Bawaan	Aaron Kwok lunga dhateng Guangzhou
Transkrip Hasil *Modelling*	Aaron Kwok lungo dhateng Guangzhou piyambak

Tabel 3.3.9.2 Perbandingan Transkrip

2. Transkrip hasil *modelling* memiliki kesalahan penulisan kata dan penambahan kata sehingga untuk kesalahan penulisan kata termasuk dalam *substitution,* penambahan kata termasuk dalam *insertion,* dan pengurangan kata termasuk *deletion* yang tidak terjadi pada contoh. Hasil kalkulasi menggnakan *Word Error Rate* dapat dilihat pada 3.3.9.3.

S = 1 (lunga != lungo) S = 1 **(3.3.9.3)**

D = 0 D = 0

I = 1 (+piyambak) I = 1

$$\boldsymbol{WER = \frac{1+0+1}{5} = 0,25}$$ Atau 25%

3.4.Desain Penelitian.

Pengaturan parameter demi mencapai tingkat WER terendah perlu dilakukan agar memaksimalkan kinerja algortima. Maka dari itu, parameter dan fungsi parameter yang akan diubah selama pembuatan model dituliskan ke tabel 3.4.1.

Algoritma	Parameter	Fungsi Parameter
GMM	Jumlah komponen gaussian	Penentu kompleksitas model
	Tolerasi konvergensi	Mengatur besaran *likelihood* sebagai batas model belajar.
	Maksimal Iterasi	Mengatur iterasi maksimal pembaharuan bobot.
TDNN	Jumlah Layer	Kedalaman representasi
	Pola Konteks	Mengatur lebar jendela waktu pada tiap layer.
	Jumlah Neuron per layer	Mengatur kompleksitas tiap layer.
	Learning rate	Mengatur besaran pembaharuan bobot.

Tabel 3.4.1. Tabel desain peneltian.

Tabel desain penelitian berisi daftar parameter dan fungsi dari tiap parameter pada algoritma model akustik yakni GMM dan TDNN.

Daftar Pustaka

Jafarzadeh Asl, H., Ghazvini Nejad, M., Edraki, A., Asgharian, M., & Partovi Nia, V. (2025). *Tiny noise-robust voice activity detector for voice assistants*. In *2025 IEEE International Workshop on Machine Learning for Signal Processing*. IEEE.

Fatehi, K., Torres Torres, M., & Kucukyilmaz, A. (2025). An overview of high-resource automatic speech recognition methods and their empirical evaluation in low-resource environments. *Speech Communication, 167*, 103151. https://doi.org/10.1016/j.specom.2024.103151

Gunarso, Buono, A., Mushthofa, & Uliniansyah, M. T. (2025). Pengembangan model akustik dengan deep neural network untuk sistem pengenalan wicara bahasa Indonesia. *Jurnal — Volume 22(1)*, 84–100. (ISSN 1693-8348; E-ISSN 2615-7128).

Jurafsky, D., & Martin, J. H. (2025). *Speech and language processing* (3rd ed., draft of August 24, 2025). Stanford University & University of Colorado Boulder.

Ilemobayo, J. A., Durodola, O., Alade, O., Awotunde, O. J., Olanrewaju, A. T., Falana, O., Ogungbire, A., Osinuga, A., Ogunbiyi, D., Ifeanyi, A., Odezuligbo, I. E., & Edu, O. E. (2024). Hyperparameter tuning in machine learning: A comprehensive review. *Journal of Engineering Research and Reports, 26*(6), 388–395. https://doi.org/10.9734/jerr/2024/v26i61188

Labied, M., Banane, M., Belangour, A., & Erraissi, A. (2022). An overview of automatic speech recognition preprocessing techniques. In *2022 3rd International Conference on Decision Aid Sciences and Applications (DASA)* (pp. 1–8). IEEE. https://doi.org/10.1109/DASA54658.2022.9765043

Labied, M., & Belangour, A. (2021). Automatic speech recognition features extraction techniques: A multi-criteria comparison. *International Journal of Advanced Computer Science and Applications (IJACSA), 12*(8), 176–182.

Malik, M., Malik, M. K., Mehmood, K., & Makhdoom, I. (2021). Automatic speech recognition: a survey. *Multimedia Tools and Applications, 80*, 9411–9457.. doi: https://doi.org/10.1007/s11042-020-10073-7

Malik, M., Malik, M. K., Mehmood, K., & Makhdoom, I. (2021). Automatic speech recognition: A survey. Multimedia Tools and Applications, 80, 9411–9457. https://doi.org/10.1007/s11042-020-10073-7

Automatic Speech Recognition: A Survey. (2020). Multimedia Tools and Applications, 79(21–22), 15329–15370. https://doi.org/10.1007/s11042-020-08822-w

Deshmukh, A. M. (2020). Comparison of Hidden Markov Model and Recurrent Neural Network in automatic speech recognition. https://doi.org/10.24018/ejers.2020.5.8.2077

Wu, F., Garcia, L. P., Povey, D., & Khudanpur, S. (2019). Advances in automatic speech recognition for child speech using factored time delay neural network. INTERSPEECH 2019.

Liu, B., Zhang, W., Xu, X., & Chen, D. (2019). Time delay recurrent neural network for speech recognition. Journal of Physics: Conference Series, 1229, 012078. https://doi.org/10.1088/1742-6596/1229/1/012078

Gunnarsson, P. D. (2019). Speech recognition for telephone conversations in Icelandic using Kaldi (Master's thesis, KTH Royal Institute of Technology).

Fathima, N., Patel, T., Mahima, C., & Iyengar, A. (2018). TDNN-based multilingual speech recognition system for low-resource Indian languages. INTERSPEECH 2018, 3197–3201.

Srivastava, B. M. L., Sitaram, S., Mehta, R. K., Mohan, K. D., Matani, P., Satpal, S., Bali, K., Srikanth, R., & Nayak, N. (2018). Interspeech 2018 low resource automatic speech recognition challenge for Indian languages. INTERSPEECH 2018.

Sithara, A., Thomas, A., & Mathew, D. (2018). Study of MFCC and IHC feature extraction methods with probabilistic acoustic models for speaker biometric applications. Procedia Computer Science, 143, 267–276. (ICACC-2018)

Ibrahim, Y. A., Odiketa, J. C., & Ibiyemi, T. S. (2017). *Preprocessing technique in automatic speech recognition for human computer interaction: An overview*. Annals. Computer Science Series, 15(1), 186–191.

Joshy, J., & Sambyo, K. (2016). A comparison and contrast of the various feature extraction techniques in speaker recognition. International Journal of Signal Processing, Image Processing and Pattern Recognition, 9(11), 99–108. https://doi.org/10.14257/ijsip.2016.9.11.10

Vijh, S., Singh, P., & Gill, M. K. (2016). Feature extraction using MFCC for speech recognition. In Fourth International Conference on Recent Trends in Communication and Computer Networks (ComNet 2016) (pp. 311–314).

Aung, M. A. A., & Pa, W. P. (n.d.). Time delay neural network for Myanmar automatic speech recognition.

Mirishkar, G. S., Yadavalli, A., & Vuppala, A. K. (n.d.). An investigation of hybrid architectures for low-resource multilingual speech recognition system in Indian context.